

POLITÉCNICO DO PORTO
INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Detecting Software Supply Chain Smells in npm Projects

Filipe Manuel Duarte Parafta

LETI
Degree in Telecommunications and Informatics Engineering

ISEP INSTITUTO SUPERIOR
DE ENGENHARIA DO PORTO

DEPARTAMENTO DE ENGENHARIA ELETROTÉCNICA
Instituto Superior de Engenharia do Porto

June, 2026

Acknowledgements

First, I would like to thank my mother for providing me with what I needed to be successful in my life. I would like to thank ISEP, more specifically the teachers of LETI, for providing a flexible schedule, allowing a large part of my time to be used for the internship project. Finally, I would like to thank Professor Isabel Azevedo for her help in finding information, for the useful observations she made during the preparation of the report, and for the information consolidation meetings, which were very useful for clarifying and bringing together ideas.

Abstract

Modern npm applications rely heavily on third-party dependencies distributed through the npm ecosystem, which increases development speed but also introduces software supply chain risks that are often difficult for developers to inspect manually. In addition to known vulnerabilities, recent research highlights a broader set of warning signs, such as dependency smells, weak link signals, and software supply chain smells, which may indicate potential maintainability or security concerns even in the absence of reported attacks or disclosed vulnerabilities.

This work addresses the problem of identifying such indicators in npm projects through a lightweight and developer-oriented analysis approach. The report first analyzes the problem context, consolidates the terminology used across related research, and organizes the considered indicators into local dependency-management indicators, registry and maintainer related indicators, and provenance and repository related indicators. It then examines the main evidence sources required for their detection, including project manifest files, lockfiles, package registry metadata, and source repository metadata.

Based on this analysis, this work presents the design and implementation of NPM Smell Analyzer, a Python-based command-line tool that combines local static inspection, selective metadata enrichment, and integration with complementary external tools to detect and report a feasible subset of supply chain risk indicators. The implemented prototype supports native detectors, Knip-based analysis, and Dirty-Waters-based checks, producing persistent output artifacts and automated test evidence. The resulting tool demonstrates the feasibility of a lightweight and developer-oriented approach for software supply chain smell detection in npm projects.

Keywords: npm, dependency smells, weak link signals, software supply chain smells, lockfiles, manifest files.

Resumo

Aplicações npm modernas dependem fortemente de dependências de terceiros distribuídas pelo ecossistema npm, o que aumenta a velocidade de desenvolvimento, mas também introduz riscos na cadeia de suprimentos de software que muitas vezes são difíceis de serem inspecionados manualmente pelos programadores. Além das vulnerabilidades conhecidas, investigações recentes destacam um conjunto mais amplo de sinais de alerta, como "dependency smells", sinais de "weak link" e "software supply chain smells", que podem indicar potenciais problemas de manutenção ou segurança, mesmo na ausência de ataques relatados ou vulnerabilidades divulgadas.

Este trabalho aborda o problema de identificar tais indicadores em projetos npm por meio de uma abordagem de análise leve e orientada ao programador. O relatório primeiro analisa o contexto do problema, consolida a terminologia usada em artigos relacionados e organiza os indicadores considerados em indicadores de gestão de dependências locais, indicadores relacionados a registros e mantenedores e indicadores relacionados à proveniência e repositórios. Em seguida, examina as principais fontes de evidência necessárias para sua detecção, incluindo ficheiros manifest, ficheiros lockfile, metadados do registo de pacotes e metadados do repositório de código-fonte.

Com base nessa análise, este trabalho apresenta a concepção e implementação do NPM Smell Analyzer, uma ferramenta de linha de comandos desenvolvida em Python que combina inspeção estática local, enriquecimento seletivo de metadados e integração com ferramentas externas complementares para detetar e reportar um subconjunto viável de indicadores de risco da cadeia de fornecimento. O protótipo implementado suporta detetores nativos, análise baseada em Knip e verificações baseadas em Dirty-Waters, produzindo artefactos persistentes e evidência de validação através de testes automatizados. A ferramenta resultante demonstra a viabilidade de uma abordagem leve e orientada a programadores para a detecção de software supply chain smells em projetos npm.

Palavras-Chave: npm, dependency smells, sinais de weak link, software supply chain smells, lockfiles, ficheiros manifest.

Contents

List of Figures	ix
List of Tables	xi
Listings	xiii
1 Introduction	1
1.1 Contextualization	1
1.2 Project Description	2
1.3 Objectives and Methodological Approach	3
1.4 Scheduling	3
1.5 Report Organization	4
2 State of the Art	5
2.1 SBOM Foundations	5
2.2 CycloneDX in React/npm Projects	7
2.2.1 Data Model and Serialization	7
2.2.2 SBOM Generation in React/npm	9
2.2.3 SBOM Analysis and Tooling	11
2.3 Dependency Trees and Vulnerability Propagation	12
2.4 Supply Chain Risk Indicators	13
2.4.1 Dependency Smells	14
2.4.2 Weak Link Signals	14
2.4.3 Software Supply Chain Smells	14
2.4.4 Risk Indicators Table	15
2.5 Related Work and Summary	15
3 Problem Analysis	19
3.1 Analysis Context	19
3.2 Scope and Problem Delimitation	21
3.3 Terminology Consolidation and Conceptual Model	22
3.4 Categories of Supply Chain Risk Indicators	24
3.5 Evidence Sources and Inputs	27
3.5.1 Local Project Files	28

3.5.2	Package Registry Metadata	29
3.5.3	Source Repository Metadata	30
3.6	Feasibility of the Proposed Approach	31
3.7	Use Case View	33
3.7.1	Use Case Description: Analyze Project	34
3.8	Requirements and Quality Attributes	35
4	Solution Conception	41
4.1	Design Goals and Requirements Basis	41
4.2	Domain Model	42
4.3	Behavioral View	45
4.3.1	Analysis Execution Flow with Local and External Smell De- tection	45
4.4	Architectural Patterns and Design Rationale	47
4.5	Smell Detection Strategy	51
4.5.1	Mapping Between Smells and Detection Mechanisms	55
4.6	Design Constraints and Trade-offs	62
5	Solution Implementation	65
5.1	Overview of the Implemented Prototype	65
5.2	Implementation Architecture	66
5.2.1	CLI and Execution Flow	67
5.2.2	Orchestration Layer	68
5.2.3	Smell Detectors	69
5.2.4	External Tool Adapters	70
5.2.5	Report Generation	71
5.3	Implementation Details	72
5.3.1	CLI and Configuration Resolution	72
5.3.2	Registry-Based Smell Dispatch	74
5.3.3	Shared Structure for Dirty-Waters-Based Smells	75
5.3.4	Knip Integration	76
5.3.5	Dirty-Waters Integration	77
5.3.6	Configuration Profiles	78
5.4	Requirements Coverage and Validation	79
5.5	Testing and Validation	83
5.5.1	Unit Tests	83
5.5.2	Integration Tests	84
5.5.3	Mock-Based Tests	85
5.5.4	Requirements Traceability	87
5.6	Repository Artifacts and Reproducibility	90
5.7	Main Development Decisions and Trade-Offs	91

6	Conclusions	93
6.1	Achieved Objectives	93
6.2	Difficulties	94
6.3	Future Work	95
6.4	Final Remarks	96
	References	97

List of Figures

1.1	Planned work schedule for the internship/project activities from February to June 2026.	4
2.1	Typical SBOM workflow for npm-based React projects: generation, quality checks, ingestion, and continuous analysis.	12
3.1	Use case diagram of the proposed solution.	34
4.1	Domain model of the proposed supply chain risk analysis tool. . . .	44
4.2	Sequence diagram for the <i>Analyze Project</i> use case.	46
4.3	Class-level view of the Strategy and Registry structures used for smell detection.	48
4.4	Class-level view of the Adapter-based integration of external analysis tools.	49
4.5	Component diagram of the proposed NPM Smell Analyzer architecture.	50
5.1	Successful execution of the unit test suite.	84
5.2	Successful execution of the integration test suite over prepared sample projects.	85
5.3	Successful execution of the mock-based test suite for external tool adapters.	87

List of Tables

2.1	NTIA SBOM minimum elements grouped into three areas.	6
2.2	Supply chain smells (S1–S5) from Dirty-Waters and proposed detection rules and mitigation considerations [1].	15
2.3	Main families of supply chain risk indicators discussed in the literature.	16
3.1	Preliminary groups of supply chain risk indicators considered in the proposed tool.	26
3.2	Feasibility of indicator groups in the first prototype of the proposed tool.	32
4.1	Comparison of alternative smell detection strategies considered for the proposed tool.	51
4.2	Detection layers adopted in the proposed tool.	52
4.3	Mapping between smell groups, adopted detection mechanisms, and implementation decisions.	54
4.4	Overview of the supply chain smells considered in this work and their support status in the prototype.	56
4.5	Detection mechanisms and implementation decisions adopted for the supply chain smells considered in the prototype.	59
5.1	Traceability between functional requirements, implementation evidence, and validation evidence.	80
5.2	Summary of automated tests used to validate the implemented tool.	82
5.3	Traceability between non-functional requirements and validation evidence.	88

Listings

2.1	Simplified CycloneDX BOM structure in JSON (illustrative).	8
2.2	Example of a bounded dependency declaration in <code>package.json</code> . . .	9
2.3	Example of dependency patterns that may increase risk in <code>package.json</code> . . .	10
2.4	Simplified excerpt of a <code>package-lock.json</code> structure (illustrative). .	10
5.1	Resolution of execution parameters in the CLI layer	73
5.2	Registry-based execution of smell detectors	74
5.3	Configuration-driven definition of Dirty-Waters-based smells	75
5.4	Core execution flow of the Knip adapter	76
5.5	Backend resolution and validation in the Dirty-Waters adapter . . .	77
5.6	Example configuration for Dirty-Waters-based smells	78
5.7	Representative unit test for a native smell detector	83
5.8	Representative integration test over a prepared sample project . . .	84
5.9	Representative mock-based test for an external tool adapter	85

Chapter 1

Introduction

This chapter presents the project developed during the second semester for the Internship/Project (PESTA) course unit, in the third year of the Bachelor’s Degree in Telecommunications and Informatics Engineering (LETI), from the Department of Electrical Engineering (DEE), at the Higher Institute of Engineering of Porto (ISEP).

The project focuses on the design and development of a software tool that is able to analyze the dependencies of npm projects and, automatically, detect and report a predefined set of supply chain smells. Supply chain smells are recurring “bad patterns” related to dependencies and their metadata that may indicate potential security or maintainability risks [1].

1.1 Contextualization

Modern web applications are increasingly dependent on a composition of large ecosystems of third-party packages and libraries. In the JavaScript ecosystem, npm projects depend on dozens of direct packages and hundreds of complementary dependencies obtained from package registries and external repositories. This dependency-driven development model accelerates product delivery but also expands the software supply chain, which can lead to a larger attack surface.

The importance of software supply chain security has become evident through large-scale incidents such as the SolarWinds breach, in which a trusted software update mechanism was abused to distribute malicious code to organizations. In this

case, attackers inserted malicious code into SolarWinds Orion software builds, allowing the compromised updates to be distributed to downstream customers through legitimate channels [2]. SolarWinds Orion is an enterprise network management software that includes performance and application monitoring and network configuration management, along with other different types of analyzing tools. Apparently, a malicious version of the binary file `solarwinds.orion.core.businesslayer.dll` was added to the SolarWinds software lifecycle, which was then signed by the legitimate SolarWinds code signing certificate. When installed, this binary calls out to a victim-specific `avsvmcloud[.]com` domain using a protocol designed to mimic legitimate SolarWinds protocol traffic. Consequently, after the initial check-in, the adversary can use the DNS response to send domains and IP addresses for interaction and traffic control commands. This case illustrates how an intrusion of this scale can propagate widely to a large number of independent victims through common means of software compilation and updates [2].

This case was one of the main reasons why supply chain security became a regulatory and governance concern. The European Union's Cyber Resilience Act, or CRA, introduces cybersecurity requirements for products with digital elements, including obligations related to vulnerability handling and incident reporting. This ultimately reinforces the need for systematic practices to ensure that the risk of components throughout the product lifecycle is not altered. According to the European Commission, the CRA entered into force on December 10, 2024. Its reporting obligations will apply from September 11, 2026, while its main obligations will apply from December 11, 2027 [3].

Focusing on application security, the OWASP Top 10:2025 explicitly recognizes these issues through the document "A03:2025 Software Supply Chain Failures," which describes failures with compromises in the process of building, distributing, or updating software. These failures often lead to potential vulnerabilities or malicious changes in code and third-party tools that use the software. These developments and concerns motivated and led to a need for lightweight, developer-oriented mechanisms that improve the visibility of project dependencies and help detect recurring risk indicators [4].

This project will address the need for dependency analysis in npm projects and will report on a predefined set of issues in the software supply chain.

1.2 Project Description

This project will address the lack of support developers have for assessing potential risks from third-party dependencies in modern npm applications. As mentioned in the project introduction, these projects often depend on a large number of direct dependencies in the npm ecosystem, which increases exposure to problems in the

software supply chain. Developers are unaware of potential malicious changes that may have been made in recent updates, and the existence of a tool that subsequently analyzes package dependencies can be useful and effective.

1.3 Objectives and Methodological Approach

The main objective of this work is to create a tool that automatically analyzes the dependencies of an npm project and reports a predefined set of supply chain risk indicators in the software supply chain. These problems are standard and recurring issues related to dependencies and package metadata that may indicate potential security or maintenance risks [1].

The goal of the proposed tool is to analyze the dependencies of npm projects and identify a predefined set of supply chain risk indicators derived from the literature. The tool is intended to support developers by providing a structured analysis of dependency-related warning signs that may affect the maintainability, transparency, or trustworthiness of third-party components.

The initial prototype will focus on npm projects and will complement traditional vulnerability-oriented tools by reporting broader supply chain risk indicators. In this way, the tool aims to improve dependency visibility and support more informed decision-making during software development and maintenance.

1.4 Scheduling

The planned schedule of the internship/project activities is presented in Figure 1.1. The work was organized into biweekly periods, covering literature review, experimentation, prototype implementation, evaluation and final report consolidation.

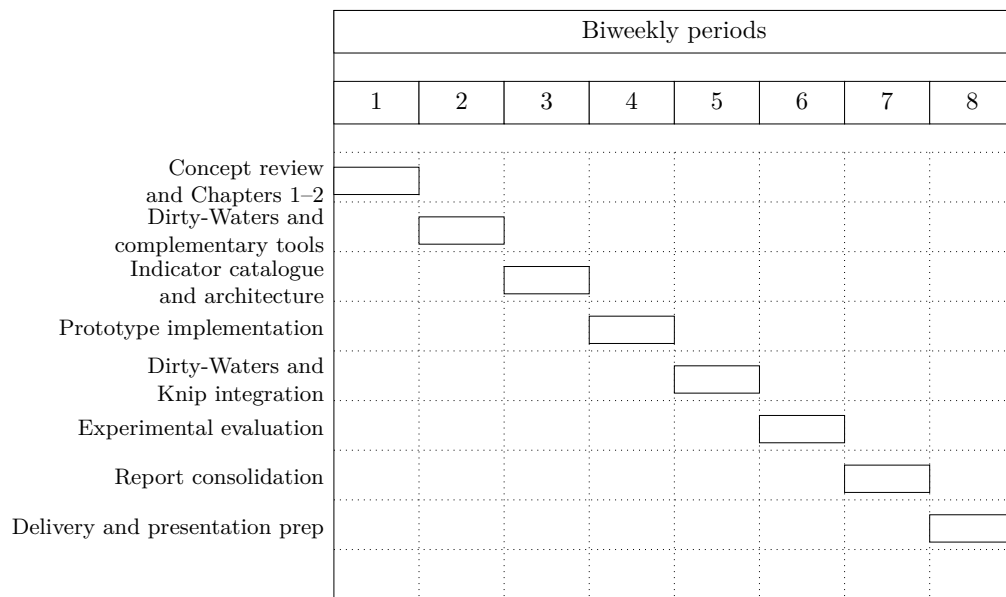


Figure 1.1: Planned work schedule for the internship/project activities from February to June 2026.

1.5 Report Organization

The document is composed of six chapters. The first provides an introduction and contextualization of the project, as well as the proposed objectives and the work schedule. The second provides an in-depth exploration of the main technology and other relevant concepts related to it. The third presents a scenario on which the project is based and an analysis of the problem. The fourth presents the solution design, including several diagrams relating to its architecture. The fifth chapter demonstrates the implementation of the solution, several code extracts, and various tests related to it. Finally, Chapter 6 presents the conclusions of the work, some of the difficulties encountered during the project, some suggestions for future work, and final assessments.

Chapter 2

State of the Art

The purpose of this chapter is to explore the main concepts associated with the problem presented previously. To this end, the chapter will explore themes associated with the problem mentioned in Chapter 1 and delve deeper into the main topic, software supply chain smells, explaining in detail and mentioning the existing categories. The objective will also be to showcase tools that support the OWASP format, in relation to the Software Bill of Materials. Finally, the propagation of vulnerabilities and their evolution through dependencies on trees within an NPM ecosystem will be discussed.

2.1 SBOM Foundations

A large part of modern systems are typically designed to depend on a vast number of third-party components, which, in turn, makes transparency of dependency a prerequisite for effective risk monitoring.

SBOM is widely described as a machine-readable record that enumerates the components used to produce a software product, capturing the supply chain relationships [5]. By accessing the inventory of existing components, an SBOM allows organizations to improve visibility into what is deployed, support vulnerability monitoring, facilitate compliance activities, and enable faster incident response when a component is encountered with some kind of problem.

The NTIA, or U.S. National Telecommunications and Information Administration, has proposed a set of minimum SBOM elements that establish a baseline for

consistency and use across tools and organizations [5]. Table 2.1 summarizes these elements in three main areas.

Table 2.1: NTIA SBOM minimum elements grouped into three areas.

Area	Description
Data Fields	Document baseline information about each component that should be tracked: supplier, component name, component version, other unique identifiers, dependency relationship, author of SBOM data, and timestamp.
Automation Support	Support automation, including automatic generation and machine-readability, to allow scaling across the software ecosystem. Data formats used to generate and consume SBOMs include SPDX, CycloneDX, and SWID tags.
Practices and Processes	Define the operations of SBOM requests, generation, and use, including frequency, depth, known unknowns, distribution and delivery, access control, and accommodation of mistakes.

With these elements, the administration verified how the SBOM, as a priority, guides software security and assurance and supply chain risk management. Following this document ensures greater security for users of the software, but also for its developers.

It is noteworthy that SBOMs are more effective when they are connected to a dependency graph that captures the different relationships between direct or transitive components. According to OWASP (a community that provides access to articles in the area of web application security) documentation, combining dependency graphs with an SBOM accurately manages vulnerabilities, which in turn speeds up incident resolution, with precise clarifications of where the components are present and how they were brought into the system [6]. This will be quite relevant for ecosystems like npm, where applications often include a huge number of dependencies that developers do not explicitly select (for example, when installing the npm environment itself on a computer, it comes with a different number of dependencies).

From a computer security perspective, the Software Bill of Materials is a concept that aligns with a broader orientation regarding the integrity of security practices throughout a software lifecycle. In this context, SBOM generation and maintenance can be seen as a practical mechanism that strengthens component inventory, traceability, and vulnerability response during the lifecycle of a software product.

2.2 CycloneDX in React/npm Projects

CycloneDX is a SBOM specification (and more specifically the Bill of Materials) created by the OWASP Foundation. Its goal is to represent software components and the relationships between them in a machine-readable way.

In practice, CycloneDX allows organizations to have a starting point for describing third-party components, which is a prerequisite for automation in dependency inventory, vulnerability response, and supply chain risk management.

The CycloneDX specification defines a data model structure (serialized in JSON or XML) to describe the different components and relevant metadata, such as identical components, versions, their dependencies, and other points or additional information that are relevant in the area of software security. This allows SBOMs to be generated and consumed by different tools in a consistent and reliable way. This specification is even more relevant in ecosystems with rapidly evolving and larger dependency graphs, such as npm, where projects often depend on different transitive packages that are not explicitly installed by the developer.

Beyond the specification, CycloneDX is supported by an ecosystem of tools that generate SBOMs from real projects and consume them for deeper analysis. For React projects, CycloneDX SBOMs can be generated from npm dependency information using dedicated generators [7]. In a separate analysis, platforms like OWASP Dependency-Track (an intelligent Component Analysis platform that allows organizations to identify and reduce risk in the software supply chain) can ingest CycloneDX SBOMs to support continuous monitoring and risk management through the software supply chain [8]. In summary, CycloneDX serves as a practical bridge between data extracted from a project's dependencies and the automated processes that handle and track supply chain risk over time.

2.2.1 Data Model and Serialization

As previously mentioned, CycloneDx defines a standardized data model to describe the software bill of materials in a way that is machine-readable and interoperable across different tools [9, 10].

This model is typically represented as a single document ("BOM") with a defined schema. This can be serialized in different formats, for example, as already mentioned, in JSON and XML. In this way, this serialization does not change the fundamental semantics of the [10] file. In practice, these serializations are useful because they allow the same information to be generated by a single tool (a SBOM generator) and consumed by another (analysis platform).

At a higher level, the CycloneDX document includes information about the BOM itself, for example, when it was generated and by which tool, a representation of the primary component to be described, a list of components (whether libraries,

services, etc.), and finally, the relationship between the component dependencies [10]. As already mentioned, this separation becomes useful because it allows tools to distinguish between the project being analyzed and third-party components that it incorporates, while also allowing a dependency representation graph that captures direct or transitive relationships.

In JSON, components are represented as structured objects that contain identifiers and attributes such as **name**, **version**, and **type**. Each component can also include additional properties and references that allow for more precise descriptions when necessary. For example, an external reference to the project repository. Relationships between dependencies are typically represented by a binding of the component identifiers.

This allows the BOM to encode the dependency graph that reflects how the components are placed and used in the final project [10].

Listing 2.1 illustrates a simplified CycloneDX JSON skeleton, showing the most common high-level sections. The exact fields and nesting depend on the version of the specification and the level of detail produced by the generator, but the overall structure remains consistent across tools [10].

```
1 {
2   "bomFormat": "CycloneDX",
3   "specVersion": "...",
4   "metadata": {
5     "timestamp": "...",
6     "tools": [ ... ],
7     "component": { ... }    // the main project / application
8   },
9   "components": [
10    { "type": "...", "name": "...", "version": "...", ... }
11  ],
12  "dependencies": [
13    { "ref": "...", "dependsOn": [ "...", "..." ] }
14  ]
15 }
```

Listing 2.1: Simplified CycloneDX BOM structure in JSON
(illustrative).

This data model is particularly valuable for ecosystems such as npm, where the set of transitive dependencies can be large and evolves quickly. By expressing component identity and dependency relationships in a standardized schema, CycloneDX enables automated downstream tasks such as vulnerability correlation, policy checks, and continuous monitoring in SBOM analysis platforms [9].

2.2.2 SBOM Generation in React/npm

In practice, generating a CycloneDX SBOM requires extracting dependency information from a project and mapping it to the CycloneDX data model. For React web applications, this generally involves parsing the ‘package.json’ file and possibly a lockfile (‘package-lock.json’) to precisely capture the resolved dependency tree, including transitive dependencies. The output will be a CycloneDX document, serialized in JSON or XML, which can be consumed by other tools consistently.

One approach used in a JavaScript ecosystem is to generate CycloneDX SBOMs directly from npm projects using dedicated generators. The package ‘@cyclonedx/cyclonedx-npm’ produces the CycloneDX SBOM by analyzing dependency information from npm and exporting a BOM that conforms to the CycloneDX specification ‘cyclonedxNpm’. This type of generator is usually suitable for local development workflows, as it can be run as part of a developer toolkit. This also allows for continuous integration (CI) pipelines, where SBOMs can be automatically generated for each build. A CI pipeline is an automated process that allows developers to combine code changes into a shared repository, automating testing and building. This process has numerous advantages such as rapid bug detection and increased software quality.

From a supply chain perspective, the likely greatest benefit of SBOM generation tools is that they standardize the dependency inventory and create an "artifact" that can be archived, monitored, and then analyzed. Producing a BOM aligned with the CycloneDX schema, these tools enable downstream processes such as vulnerability correlation, policy checks, and historical tracking of changes in the dependency set over time [9, 10].

In short, SBOM generation becomes a fundamental step to support continuous supply chain risk management in React applications.

In npm-based projects, dependency declarations in `package.json` typically use semantic versioning ranges (e.g., caret `^` and tilde `~`), which allow controlled updates within a compatible range. While flexible ranges can simplify maintenance, overly permissive ranges (e.g., wildcards) may introduce instability and increase the chance of unintentionally pulling unexpected versions [11, 12].

Listing 2.2 shows a typical `package.json` with bounded version ranges. Listing 2.3 illustrates patterns that are commonly considered higher-risk in a supply-chain context, such as floating versions and non-registry sources.

```
1 {  
2   "name": "react-app",  
3   "private": true,  
4   "dependencies": {  
5     "react": "^18.2.0",  
6     "react-dom": "^18.2.0",
```

```

7     "axios": "^1.7.0"
8   },
9   "devDependencies": {
10     "typescript": "^5.4.0",
11     "vite": "^5.0.0"
12   }
13 }

```

Listing 2.2: Example of a bounded dependency declaration in `package.json`.

```

1 {
2   "name": "react-app",
3   "private": true,
4   "dependencies": {
5     "react": "*", // wildcard (floating)
6     "some-lib": "latest", // floating tag
7     "custom-lib": "git+https://github.com/org/custom-lib.git#main"
8   },
9   "scripts": {
10    "postinstall": "node scripts/setup.js" // runs after install
11  }
12 }

```

Listing 2.3: Example of dependency patterns that may increase risk in `package.json`.

To make dependency analysis reproducible, npm projects commonly rely on `package-lock.json`, which records the exact dependency tree resolved during installation. This enables subsequent installs to reproduce the same dependency graph, even if upstream packages evolve [12]. In addition, security auditing workflows depend on this resolved view: by default, `npm audit` requires a lockfile (or `shrinkwrap`) and may yield different results without it [13, 14].

Listing 2.4 shows an illustrative excerpt of a lockfile structure, where the resolved dependency versions and metadata are captured.

```

1 {
2   "name": "react-app",
3   "lockfileVersion": 3,
4   "packages": {
5     "": {
6       "dependencies": {
7         "react": "^18.2.0"
8       }
9     },

```

```
10     "node_modules/react": {  
11       "version": "18.2.0",  
12       "resolved": "https://registry.npmjs.org/react/-/react  
        -18.2.0.tgz",  
13       "integrity": "sha512-..."  
14     }  
15   }  
16 }
```

Listing 2.4: Simplified excerpt of a `package-lock.json` structure
(illustrative).

SBOM generation can be integrated at different stages of the development lifecycle. During local development, SBOMs can be generated on-demand to provide immediate visibility into direct and transitive dependencies. In CI, SBOM generation can be automated as a build step, ensuring that each release or build artifact is accompanied by an SBOM that reflects the exact dependency set resolved at build time [10].

For npm-based React projects, best results are obtained when the generator uses the lockfile, since it captures the precise versions actually installed and the complete resolved dependency tree. This reduces ambiguity compared to relying only on semver ranges declared in `package.json`. Tools such as `@cyclonedx/cyclonedx-npm` are designed for this workflow and can be executed in both local and CI environments [7].

Although SBOM generators provide a standardized inventory, the quality and completeness of the produced SBOM depends on the available input data and on how the generator interprets project metadata. For example, missing or inconsistent lockfiles may reduce the ability to precisely capture transitive dependencies. In addition, different generators may vary in the depth of metadata included (e.g., external references, license information, or dependency edges), even when they target the same CycloneDX specification version [10]. These considerations motivate the need to evaluate generation tools not only by output format support (CycloneDX JSON/XML), but also by reproducibility, completeness, and ease of integration into developer workflows.

2.2.3 SBOM Analysis and Tooling

Generating an SBOM is only the first step in supply chain risk management. SBOM analysis refers to the set of processes and tools that consume a machine-readable SBOM (CycloneDX) and derive actionable information from it, such as identifying affected components, tracking dependency changes over time, enforcing policies (security and licensing), and supporting incident response workflows [10, 9]. In

practice, analysis can be performed either through centralized platforms that continuously monitor projects, or through local tools that validate, diff, and transform SBOM artifacts.

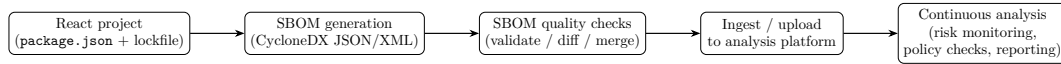


Figure 2.1: Typical SBOM workflow for npm-based React projects: generation, quality checks, ingestion, and continuous analysis.

2.3 Dependency Trees and Vulnerability Propagation

OWASP Dependency-Track is a platform designed for continuous component analysis based on SBOMs. It ingests SBOMs (including CycloneDX) and provides capabilities to identify and reduce risk in the software supply chain, supporting use cases that go beyond one-off scans [8, 15]. Dependency-Track emphasizes lifecycle tracking: projects can be monitored over time, enabling teams to observe how component inventories evolve and to prioritize remediation actions as new vulnerability intelligence becomes available [16].

From a workflow perspective, a typical setup consists of generating a CycloneDX SBOM during local development or CI and then uploading it to Dependency-Track, which maintains the project’s component inventory and correlates it with vulnerability and policy information [16]. This approach is particularly relevant for npm-based applications, where transitive dependencies may be numerous and frequently updated: by anchoring analysis on an SBOM, the platform can provide a consistent view of what is actually present in a given build or release.

In addition to centralized platforms, the CycloneDX ecosystem includes local tools that support SBOM analysis tasks such as validation, merging, diffs, format conversion, and signing. These tools can be used as part of a developer workflow or integrated into CI pipelines to ensure SBOM artifacts are well-formed and comparable across versions [17]. Such tooling complements platforms like Dependency-Track by improving the quality and consistency of SBOMs before ingestion and by enabling lightweight analysis (e.g., diffing SBOMs between releases) during development.

The npm ecosystem enables rapid development by promoting extensive reuse of third-party libraries. However, this reliance creates a systemic security concern: vulnerabilities introduced in upstream packages can propagate to a large number of downstream projects through transitive dependencies. Liu et al. investigate this phenomenon and argue that accurately understanding vulnerability propagation in npm requires modeling dependency trees using npm-specific dependency resolution rules, since analyses that only consider direct dependencies or approximate transitive dependencies may produce incorrect results [18].

To address this challenge, the authors propose a knowledge-graph-based approach to resolve dependency relations precisely at ecosystem scale. They construct a dependency-vulnerability knowledge graph (DVGraph) covering the npm ecosystem and introduce DTResolver, an algorithm that statically resolves dependency trees and transitive vulnerability propagation paths while considering the official npm resolution rules [18]. Based on this infrastructure, the paper performs an ecosystem-wide empirical study on how vulnerabilities propagate across dependency trees and how such propagation evolves over time.

Beyond characterization, the work also discusses actionable mitigation strategies for different stakeholders. As an example, the authors present DTReme, a dependency-tree-based remediation method, and report that it achieves better performance than the default remediation mechanism provided by `npm audit fix` [18]. These results reinforce a key insight for dependency analysis in JavaScript projects: supply chain risk assessment and remediation should be grounded on the resolved dependency tree, not solely on declared dependencies.

This paper is directly relevant to the present project for two reasons. First, it motivates the importance of reconstructing dependency trees accurately when analyzing React/npm projects, since risk indicators (including supply chain smells) may stem from transitive dependencies. Second, it provides evidence that ecosystem-specific resolution rules influence the correctness of analysis results, which is essential when designing automated tooling for dependency and supply chain risk assessment [18].

2.4 Supply Chain Risk Indicators

While SBOM generation and vulnerability scanning are important components of software supply chain security, they do not capture every type of risk associated with third-party dependencies. In particular, some relevant warning signs are not confirmed vulnerabilities, but rather observable indicators that may suggest increased uncertainty, reduced transparency, or weaker trust in external components. In the literature, these indicators are discussed under different terms, including *dependency smells*, *weak link signals*, and *software supply chain smells* [19, 20, 1].

Although these terms are not strictly identical, they describe related perspectives on the same general problem: how to identify warning signs in third-party dependencies before they necessarily become concrete failures or incidents. The main differences lie in the type of evidence considered and in the layer of analysis emphasized by each work. The following subsections summarize these three perspectives and their relevance to the present project.

2.4.1 Dependency Smells

Jafari et al. introduce the concept of *dependency smells* in JavaScript projects to describe recurring dependency-management issues that may negatively affect maintainability, security, or dependency stability [19]. In this perspective, the focus is primarily on the way a project declares and manages its dependencies locally, especially through files such as `package.json` and the associated lockfile.

The authors identify several dependency smells, including pinned dependencies, URL-based dependencies, restrictive constraints, permissive constraints, the absence of a lockfile, unused dependencies, and missing dependencies [19]. These smells are particularly relevant because they can often be detected through lightweight static inspection of project files and code usage, without requiring extensive ecosystem-wide metadata.

For React/npm projects, this perspective is especially useful because many dependency-related warning signs originate directly from project-level dependency practices. As a result, dependency smells provide a practical foundation for local analysis features in the proposed tool.

2.4.2 Weak Link Signals

Zahan et al. use the term *weak link signals* to describe observable conditions that may expose npm packages to a higher risk of software supply chain attacks [20]. In contrast to dependency smells, which focus mainly on dependency declarations in the consuming project, weak link signals rely more strongly on package-registry and maintainer-related metadata.

The weak link signals identified in the study include expired maintainer domains, the presence of installation scripts, unmaintained packages, packages with too many maintainers, packages with too many contributors, and overloaded maintainers [20]. These indicators do not prove that a package is malicious, but they may reveal structural weaknesses in the way packages are published, maintained, or governed within the npm ecosystem.

This perspective is relevant to the present project because it broadens the analysis beyond local project files. It shows that some dependency-related warning signs only become visible when package metadata and maintainer-related information are considered as part of the supply chain assessment.

2.4.3 Software Supply Chain Smells

Liu et al. introduce the term *software supply chain smells* to refer to recurring bad patterns in dependencies and their associated metadata that may indicate potential security or maintainability risks, even when no known vulnerability has yet been reported [1]. This perspective is implemented in the Dirty-Waters tool, which

combines dependency manifests, package-registry metadata, and source-repository metadata in order to detect such signals.

Dirty-Waters emphasizes smells related to provenance, transparency, and release traceability. Examples include missing or inaccessible source code links, inaccessible release tags, deprecated dependencies, dependencies whose source repository is a fork, and packages without sufficient provenance-related information [1]. These smells are not equivalent to confirmed vulnerabilities; rather, they indicate conditions that may hinder verification, reduce trust, or make the software supply chain more difficult to assess.

Table 2.2 summarizes the main supply chain smells discussed by Dirty-Waters, together with their corresponding evidence type and high-level detection logic.

Table 2.2: Supply chain smells (S1–S5) from Dirty-Waters and proposed detection rules and mitigation considerations [1].

ID	Smell	Type	Category	Detection rule (high-level)	Finding the smell	Fixing the related problem
S1	No source code link / source inaccessible	Cross-check	Provenance & Transparency	Extract repository/source link from registry metadata. Mark as smell if the link is missing, malformed, or the URL/repository is not accessible (e.g., returns 404/401) or does not resolve to a valid source location.	Med	Med
S2	Release tag is inaccessible	Repository-based	Release Integrity	For the dependency version, check whether a corresponding repository tag or release reference exists and is accessible. Mark as smell if the tag is missing or cannot be retrieved.	High	Med
S3	Package is marked as deprecated	Registry-based	Maintenance & Support	Query registry metadata for the package. Mark as smell if the package is flagged as deprecated (with a deprecation message).	Low	Low
S4	Source code repository is a fork	Repository-based	Provenance & Transparency	Identify the repository hosting the package source. Mark as smell if the repository is labeled as a fork by the hosting platform.	Low	Med
S5	Package without provenance	Registry-based	Provenance & Transparency	Check whether provenance-related metadata needed to identify maintainers/origin is missing or insufficient (e.g., missing maintainer/author fields or missing repository reference). Mark as smell if provenance cannot be established from registry metadata alone.	Med	Med

2.4.4 Risk Indicators Table

Table 2.3 summarizes the three main families of supply chain risk indicators discussed in the literature, together with representative examples and their main evidence sources.

As shown in Table 2.3, the literature converges on the idea that third-party dependencies may exhibit observable warning signs even when no confirmed vulnerability has yet been reported. The main differences lie in the type of evidence required and in the analysis layer emphasized by each work.

2.5 Related Work and Summary

The topics covered in this chapter connect to three complementary research and practice areas: (i) SBOM standards and tooling, (ii) ecosystem-scale analysis of dependency-driven risk in npm, and (iii) lightweight indicators of supply chain risk beyond known vulnerabilities.

Regarding SBOMs, CycloneDX provides a standardized and interoperable data model for representing components and dependency relationships, enabling SBOM artifacts to be generated and consumed across different tools [9, 10]. The existence of analysis platforms such as OWASP Dependency-Track further demonstrates how

Table 2.3: Main families of supply chain risk indicators discussed in the literature.

Indicator family	Representative indicators	Main evidence source	Origin in literature
Dependency smells	Pinned dependencies, URL dependencies, restrictive or permissive constraints, missing lockfile, unused dependencies, and missing dependencies	Local project files (<code>package.json</code> , lockfile, code usage)	Jafari et al.
Weak link signals	Expired maintainer domain, installation scripts, unmaintained package, too many maintainers, too many contributors, and overloaded maintainer	Package-registry and maintainer-related metadata	Zahan et al.
Software supply chain smells	Missing or inaccessible source code link, inaccessible release tag, deprecated dependency, forked repository, and missing provenance information	Package-registry metadata and source-repository metadata	Liu et al. / Dirty-Waters

SBOMs can be operationalized for continuous monitoring and risk management once they are generated as part of a development or CI workflow [8, 16]. In addition, guidance such as the OWASP Dependency Graph SBOM cheat sheet emphasizes the value of combining SBOM artifacts with dependency graphs to improve vulnerability management and incident response by clarifying direct and transitive relationships [6].

From a research perspective, Liu et al. study vulnerability propagation in the npm ecosystem by resolving dependency trees while explicitly considering npm-specific dependency resolution rules. Their results highlight that correct modeling of transitive dependencies is essential to reason about vulnerability propagation paths and to support remediation strategies at scale [18]. This complements SBOM-based approaches by providing evidence that ecosystem-specific resolution semantics directly influence the correctness of dependency analysis and, therefore, the accuracy of risk assessment.

Finally, vulnerability databases and auditing tools tend to focus on known vulnerabilities, but supply chain risk can also emerge from patterns related to provenance, maintenance, and transparency. Dirty-Waters introduces software supply chain smells as recurring bad patterns that can indicate potential risks even in the absence of reported vulnerabilities, and proposes a tool-driven approach that combines registry and repository metadata to detect such signals [1]. This line of work motivates an additional layer of developer-oriented diagnostics that complements SBOM generation and vulnerability scanning.

Overall, the reviewed literature and tooling suggest that effective supply chain risk assessment in React/npm projects benefits from (i) a standardized representation of dependencies (e.g., CycloneDX SBOMs), (ii) accurate reconstruction of resolved dependency trees, and (iii) detection of higher-level risk indicators derived from supply chain metadata. These observations support the motivation for the present project, which aims to analyze the dependencies of npm projects and report a predefined set of supply chain smells, providing actionable insights that complement traditional vulnerability-oriented tooling.

Chapter 3

Problem Analysis

This chapter provides a detailed analysis of the problem surrounding software supply chain risks in the npm ecosystem. It begins by establishing the analysis context and delimiting the scope of the project. The chapter then consolidates the terminology used throughout this work and organizes the supply chain risk indicators into distinct categories based on current research. Furthermore, it examines the primary evidence sources required for detection and assesses the feasibility of the proposed approach. Finally, the chapter defines the functional and non-functional requirements, including the use case view and the quality attributes that the system must satisfy.

3.1 Analysis Context

Modern npm applications are typically developed using a huge ecosystem of third-party dependencies, usually distributed via npm. In practice, even small projects can depend on dozens of direct dependencies and hundreds of transitive dependencies, often without the developer's knowledge. This creates a software supply chain that is difficult for those involved to manually inspect each one. As discussed in Chapter 2, this dependency-driven model promotes faster development and greater code reuse, but also increases software vulnerability and creates security problems that originate from outside the project's own source code.

The first difficulty is that developers often mention or reference dependencies through manifest files such as `package.json`, when the actual software supply chain

of the project is determined by the resolved dependency tree. In npm-based ecosystems, this dependency tree differs significantly from what is intuitively inferred from declared direct dependencies, especially when talking about different versions, transitive dependencies, and even ecosystem-specific resolution rules [12, 18]. Previous work has shown that the accuracy of risk analysis of related dependencies in npm requires a correct reconstruction of the dependency tree, since vulnerabilities and other problems can propagate through transitive relationships instead of just direct dependencies [18].

A second difficulty is that conventional dependency auditing workflows are mostly focused on known vulnerabilities. Even though vulnerability databases and npm auditing tools are important, they only cover part of the problem. [13, 14] A dependency may not have any published vulnerabilities and still exhibit characteristics that justify concern, such as being deprecated, poorly maintained, inconsistent links to its base repository, or declared in a way that makes updating and verifying the dependency difficult. For this reason, recent studies have begun to emphasize a larger catalog of categories of danger signals. Examples include dependency smells, weak link signals, and software supply chain smells [19, 20, 1]. These concepts do not describe a confirmed attack or defect; instead, they describe observable patterns that may indicate a potential risk, reduced transparency, or weak trust in the supply chain.

This distinction is particularly relevant for the present work. The goal of the project is not to determine whether a dependency is malicious, nor to replace vulnerability scanners. Instead, the goal is to identify indicators of potential future problems in npm project dependencies, so that developers can inspect them earlier and make more informed maintenance decisions. In this sense, the problem addressed in this report is closer to early warning and decision support than to definitive security diagnosis. This perspective is aligned with the Dirty-Waters view of supply chain smells as recurring bad patterns that may indicate potential security or maintainability risks even when no vulnerability has yet been reported [1].

The literature also suggests that these indicators can emerge from different layers of evidence. Some are observable locally in the project itself, for example through dependency declarations, lockfiles, or mismatches between declared and used packages [19]. Others depend on external metadata, such as package registry information or source repository availability [20, 1]. This means that the analysis problem is not limited to parsing a single file; rather, it involves correlating multiple sources of evidence that describe different aspects of a dependency’s lifecycle, maintenance status, and provenance. The practical challenge is therefore to turn this heterogeneous information into a form that is understandable and useful for developers working in real npm projects.

From an engineering perspective, another important issue is usability. Some

existing research tools demonstrate that supply chain smells can be detected, but they may require multiple external services, non-trivial setup, or relatively heavy analysis workflows. While such approaches are valuable for research and large-scale studies, they are not always ideal for lightweight developer-oriented use in local environments or continuous integration pipelines [1, 21]. Therefore, there is a need for a solution that preserves the analytical value of these indicators while remaining practical, understandable, and sufficiently lightweight for routine use.

Based on this context, the problem addressed in this chapter can be stated as follows: developers of npm applications lack a lightweight and structured mechanism to identify, consolidate, and explain supply chain risk indicators derived from dependency manifests, resolved dependency data, and selected external metadata. As a consequence, relevant warning signs may remain unnoticed until they evolve into concrete maintainability or security problems. The purpose of the following sections is to analyze this problem in detail, define its scope, and establish the requirements and conceptual basis for the proposed tool.

3.2 Scope and Problem Delimitation

The present work focuses on applications that use the npm ecosystem for dependency management. In practical terms, this means that the primary inputs considered by the proposed tool are the project manifest file, namely `package.json`, and, when available, the lockfile `package-lock.json`. In npm-based projects, dependency declarations are specified in `package.json`, while the lockfile records the exact dependency tree generated during installation, allowing subsequent installs to reproduce the same resolved tree [12]. This makes these files the most relevant local sources for a lightweight analysis of dependency-related risk indicators.

The scope of the analysis includes both direct and transitive dependencies. This decision is motivated by previous work showing that relevant security and maintenance issues in npm may propagate through transitive relations and that correct reasoning about dependency-related risk requires reconstructing dependency trees according to npm-specific resolution rules [18]. Therefore, limiting the analysis only to direct dependencies would provide an incomplete view of the effective software supply chain of an npm project.

The proposed tool is intended to detect *indicators* of potential software supply chain problems rather than to confirm attacks, prove malicious intent, or replace vulnerability scanners. In this sense, the focus is on observable warning signs such as problematic dependency declarations, dependency-management inconsistencies, deprecation status, provenance-related concerns, and selected metadata conditions that may reduce trust in third-party components [19, 20, 1]. This delimitation is important because it distinguishes the present work from tools whose primary goal

is to report already disclosed vulnerabilities, such as conventional audit workflows [13, 14].

At the local-project level, the scope includes indicators that can be inferred from manifest files, lockfiles, and dependency usage consistency. Examples include dependency declaration patterns, absence of a lockfile, mismatches between declared and used dependencies, and other issues related to dependency management practices [19, 12]. At the external-metadata level, the scope includes selected indicators derived from package registry metadata and, when relevant, source repository metadata, since prior work has shown that signals such as deprecation, provenance limitations, repository accessibility, and maintainer-related metadata may reveal additional supply chain concerns that are not visible from local files alone [20, 1].

Several aspects are intentionally left outside the scope of this stage. First, the tool is not intended to perform dynamic analysis or runtime monitoring of installed packages. Second, it does not aim to execute dependency code, validate exploitability, or determine whether a package is definitively malicious. Third, the initial scope is limited to projects in the npm ecosystem, which means that other package managers or ecosystems are not primary targets of the first version. Finally, although the generated analysis may support developer decision-making, fully automated remediation is outside the current scope and should be considered future work.

This delimitation reflects a practical engineering trade-off. A broad enough scope is needed to capture meaningful supply chain risk indicators from both local and external sources, but the scope must also remain narrow enough to support a lightweight, understandable, and reproducible analysis workflow suitable for development environments and continuous integration contexts. By clearly restricting the problem in this way, the present work establishes a feasible basis for the conception of a developer-oriented tool that complements, rather than replaces, existing SBOM and vulnerability-analysis approaches [1].

3.3 Terminology Consolidation and Conceptual Model

It is worth noting that in Chapter 2, only the Dirty-Waters tool and supply-chain smells were mentioned, but related works, already discussed in this chapter, use a different term for warning signals in software supply chains. The other two projects studied report different aspects of the same problem. The report "Dependency Smells in JavaScript Projects" uses the term "dependency smells" to describe recurring dependency monitoring problems in JavaScript projects that can have negative consequences on maintainability, security, or dependency stability.

In the project "What are Weak Links in the npm Supply Chain?", the term "weak link signals" is used to describe conditions based on observable metadata that

expose npm packages to a higher risk of supply chain attacks. To recap, Dirty-Waters introduces the term *software supply chain smells* to refer to bad patterns in dependencies and the metadata associated with them that indicate a potential security problem, now or in the future [1].

Even though the terms are not strictly identical, conceptually they are related. The three perspectives describe observable indicators associated with third-party dependencies that may warrant a deeper inspection, even if there is no confirmation of a vulnerability or an already reported attack. The difference remains, mainly, in the analysis layer and the type of evidence presented in each work. In "Dependency Smells in JavaScript Projects", the focus is on how the project declares and manages dependencies locally, for example through package constraints, lockfile usage, or mismatches between declared and used packages [19]. In "What are Weak Links in the npm Supply Chain?", the focus shifts to the metadata log of packages and maintainer-related signals that may expose the supply chain to opportunistic attacks [20].

Finally, in Dirty-Waters, the focus is broader and combines file dependencies, registry metadata, and source repository metadata that identifies patterns related to transparency and maintenance status [1].

For the purposes of this work and current idea, the different perspectives are treated as complementary rather than contradictory. Therefore, this report will adopt the following operational definition: a *supply chain risk indicator* would be any observable pattern, derived from the dependencies of manifests, lockfiles, package metadata, or even repository metadata, that may in some way aggressively increase the maintainability or security risk associated with a third-party dependency. This definition is intentionally broader than the term "vulnerability." The goal is for the tool not to confirm that a dependency is compromised or that it has a virus or attack, but rather to identify warning signs that may require the developer's attention.

Based on this definition, the conceptual model used in this project distinguishes three main categories of indicators. The first category comprises *local dependency-management indicators*, which are observable from project files such as `package.json`, `package-lock.json`, or from consistency checks between declared and used dependencies. This category covers patterns close to the notion of dependency smells described by Jafari et al. [19]. The second category comprises *registry- and maintainer-related indicators*, which depend on package-registry metadata and include signals such as deprecation or maintainer-related weaknesses, in line with the weak-link perspective of Zahan et al. [20]. The third category comprises *provenance and repository-related indicators*, which require reasoning about links between published packages and their source repositories, as emphasized by Dirty-Waters [1].

This conceptual consolidation is useful for two reasons. First, it allows the present work to integrate evidence from different strands of literature without forcing

all indicators into a single pre-existing terminology. Second, it provides a structured basis for the conception of the proposed tool, whose role is to collect, organize, and explain these indicators in a unified and understandable way. In other words, the tool is not centered on a single family of smells, but on a broader developer-oriented model of supply chain risk indicators that can be derived from multiple evidence sources.

3.4 Categories of Supply Chain Risk Indicators

Based on what was seen in chapter 2, the proposed tool should not rely on a single type of signal. Instead, it should support a catalog of indicators derived from the different sources highlighted and grouped according to the software supply chain layer in which they are observed. This is important because some indicators are visible directly in the project files, while others only appear when the data of a dependency is correlated with the package-registry or the source-repository metadata [19, 20, 1].

The first group comprises local dependency-management indicators. These are indicators that can be inferred from files such as `package.json` and `package-lock.json`, or from consistency checks between declared dependencies and dependencies actually used. This group includes patterns such as restrictive or overly permissive dependency declarations, pinned versions, URL-based dependencies, the absence of a lockfile, mismatches between the declared package and the one actually used, and similar dependency-management problems. These indicators are particularly relevant because they can often be detected with a lightweight analysis that provides an immediate view of practices adopted in the project regarding dependencies.

The second group comprises *registry- and maintainer-related indicators*. These indicators depend on metadata obtained from the package registry and describe conditions that may not be visible from local project files alone. Examples include deprecated packages, weak provenance information, maintainer-related weaknesses, or other metadata conditions that may reduce trust in a dependency or increase exposure to supply chain attacks [20, 1]. This group is important because many supply chain concerns are linked not only to how a project declares dependencies, but also to how those dependencies are published, maintained, and identified in the ecosystem.

The third group comprises *provenance and repository-related indicators*. These require the analysis of links between the published package and its associated source repository. Examples include missing or inaccessible source links, inaccessible release tags, repository forks, or other inconsistencies that make it harder to verify the origin and integrity of a dependency [1]. Such indicators are especially relevant in

the context of supply chain trust, since they affect transparency and the developer’s ability to inspect the package origin or validate its release history.

For the first version of the tool, this catalogue should be treated as preliminary rather than exhaustive. The purpose is not to support every possible signal reported in the literature, but to identify a feasible and meaningful subset of indicators that can be implemented in a lightweight way and that provide useful feedback to developers. In practice, priority should be given to indicators that satisfy at least one of the following conditions: they are well supported in the literature, they are observable with available data sources, or they are likely to produce understandable and actionable findings in npm projects.

This preliminary catalogue also supports a practical design principle for the proposed tool: each finding should be reported not only as a detected issue, but also together with its category, evidence source, and interpretation. In this way, the tool can present indicators in a structured and comprehensible manner, instead of producing an isolated list of warnings. This is particularly important in a developer-oriented setting, where the value of the analysis depends not only on detection coverage, but also on the clarity with which the detected indicators are explained.

Table 3.1 summarizes the three preliminary groups of supply chain risk indicators considered in this work, the main sources of evidence required to detect them, and their implications for the proposed tool. This grouping helps clarify that the analysis is not limited to local project files, but also depends on selected external metadata when broader supply chain concerns are considered.

Table 3.1: Preliminary groups of supply chain risk indicators considered in the proposed tool.

Group	Main evidence source	Examples of indicators	Implications for the tool
Local dependency-management indicators	Project files (<code>package.json</code> , <code>package-lock.json</code>) and local consistency checks	Pinned dependencies, permissive/restrictive version constraints, URL dependencies, missing lockfile, declared-but-unused or missing dependencies	Can be detected with lightweight static analysis; form the core local analysis layer of the tool.
Registry- and maintainer-related indicators	Package registry metadata	Deprecated packages, weak provenance information, maintainer-related weaknesses, metadata suggesting reduced trust	Require external metadata collection and enrichment beyond local project files.
Provenance and repository-related indicators	Source repository metadata and cross-checks with package metadata	Missing source links, inaccessible release tags, forked repositories, repository inconsistencies	Require correlating registry and repository information; increase transparency and trust analysis capabilities.

As shown in Table 3.1, the three groups differ mainly in the origin of the evidence and in the complexity of the corresponding analysis. Local dependency-management indicators are the most accessible to detect, since they rely on files already present in the project. By contrast, registry-related and provenance-related indicators require external metadata enrichment and cross-source correlation, which increases analytical coverage but also raises implementation complexity.

3.5 Evidence Sources and Inputs

The identification of supply chain risk indicators in npm projects depends on the availability of multiple evidence sources rather than on a single project file. This follows directly from the nature of the problem discussed in the previous sections: some indicators can be observed locally in dependency manifests and lockfiles, while others only become visible when project data is enriched with package-registry or source-repository metadata [19, 20, 1].

At the project level, dependency information is primarily declared in `package.json`, where packages are listed as `dependencies` or `devDependencies`, together with their associated version requirements [22]. However, this file alone does not fully represent the effective software supply chain of the application. In npm-based projects, the resolved dependency tree is captured more precisely by the lockfile, which describes the exact tree generated during installation and allows subsequent installs to reproduce the same dependency set [12]. As a result, both manifest and lockfile information are relevant inputs for the proposed tool.

Nevertheless, not all relevant indicators can be derived from local files alone. Some require metadata obtained from the package registry, such as package status, provenance-related information, or maintainer-associated signals [20, 1]. Others require metadata associated with the package source repository, especially when the analysis concerns transparency, release traceability, or the consistency between a published package and its declared source [1]. This means that the tool must be conceived as an analysis workflow that combines local project inspection with selected external metadata enrichment.

This distinction is also important from an engineering perspective. Local project files are immediately available, deterministic, and lightweight to process, which makes them suitable for the core analysis workflow. External metadata, by contrast, may increase analytical coverage but also introduce additional complexity, such as network dependence, missing information, or the need to correlate various sources. For this reason, the evidence model of the proposed tool should clearly distinguish between locally derived findings and externally enriched findings, while still presenting both in a unified and understandable way to the developer.

Based on these considerations, the present section organizes the main evidence sources required by the proposed approach into three groups: local project files, package-registry metadata, and source-repository metadata. The following subsections describe the role of each group and its relevance for the analysis of supply chain risk indicators in npm applications.

3.5.1 Local Project Files

Local project files constitute the primary and most immediately accessible evidence source for the proposed tool. In npm-based projects, dependency declarations are specified in `package.json`, where packages are listed under `dependencies` or `devDependencies`, together with their version requirements [22]. This file provides the initial view of the software supply chain intended by the developer and is therefore a natural starting point for the analysis.

However, `package.json` alone does not fully describe the effective dependency set of the project. In practice, dependency resolution may introduce transitive packages and concrete installed versions that are not explicit in the manifest itself. For this reason, the lockfile, namely `package-lock.json`, is also a critical local input. According to the npm documentation, the lockfile records the exact dependency tree generated during installation, allowing future installs to reproduce the same resolved dependency set [12]. This makes the lockfile particularly relevant when the analysis requires reasoning about the effective software supply chain rather than only about declared direct dependencies.

From the perspective of the proposed tool, local project files support the detection of indicators that can be derived without requiring external metadata enrichment. These include dependency declaration patterns, version-constraint issues, the absence of a lockfile, and inconsistencies between declared and effectively used dependencies [19, 12]. Such indicators are especially valuable in the first version of the tool because they can be detected through lightweight static analysis and provide immediate feedback to developers.

Another advantage of relying on local files is reproducibility. Since `package.json` and `package-lock.json` are part of the project itself, they are deterministic inputs that can be analyzed consistently across developer environments and continuous integration workflows. This makes them suitable as the core evidence source of the analysis process, even when the overall approach is later complemented with selected external metadata sources.

For these reasons, local project files form the foundation of the proposed analysis workflow. They provide the baseline evidence required to reconstruct dependency declarations, inspect resolved dependency data, and identify a first layer of supply chain risk indicators before any additional enrichment step is performed.

3.5.2 Package Registry Metadata

Although local project files provide the foundation for dependency analysis, they do not expose all the information required to identify broader software supply chain risk indicators. Some relevant signals are only visible through metadata published in the package registry, since this metadata describes properties of the package that go beyond the dependency declarations present in the consuming project. In the npm ecosystem, registry metadata can be inspected through the registry itself or through commands such as `npm view`, which expose published package information [23].

From the perspective of the proposed tool, package registry metadata is particularly important because it enables the detection of indicators related to package status, provenance, and maintainer-associated conditions. For example, a package may be marked as deprecated in the registry, even though this information is not evident from the local dependency declaration alone. Likewise, registry metadata may reveal missing or weak provenance information, maintainer-related signals, or other package-level conditions that influence the degree of trust that developers can reasonably place in a dependency [20, 1].

This source of evidence is also important because it complements, rather than replaces, local analysis. A dependency may appear syntactically well-formed in `package.json` and still present warning signs when its registry metadata is examined. In this sense, registry metadata extends the visibility of the tool from the project’s local view of dependencies to the ecosystem-level context in which those dependencies are published and maintained [20].

At the same time, registry-based analysis introduces additional engineering considerations. Unlike local files, registry metadata depends on external availability and may be incomplete, missing, or subject to change over time. This means that findings derived from this source should be clearly distinguished from purely local findings, while still being integrated into the same reporting workflow. For the purposes of the proposed tool, registry metadata should therefore be treated as an enrichment layer that expands analytical coverage without replacing the core local analysis stage.

For these reasons, package registry metadata constitutes the second major evidence source of the proposed approach. It supports the identification of supply chain risk indicators that are not directly observable in project files and helps contextualize dependencies in terms of maintenance status, provenance, and ecosystem-level trust signals.

3.5.3 Source Repository Metadata

A third relevant evidence source for the proposed tool is the metadata associated with the source repository of a dependency. While local project files and package-registry metadata provide important information about dependency declarations, resolved versions, and package-level publication data, they do not always provide enough visibility into the actual origin and traceability of the package. In the npm ecosystem, package metadata may include repository references, and such information can be inspected through registry queries, for example using commands such as `npm view` [23].

Source repository metadata is particularly relevant when the analysis concerns transparency, release traceability, and provenance. A published package may declare a repository link, but this link may be missing, inaccessible, inconsistent with the published version, or otherwise insufficient to support trust in the package origin. Likewise, repository-level conditions such as inaccessible release tags or forked repositories may affect a developer’s ability to validate where a package comes from and how it relates to its claimed source history [1].

From the perspective of the proposed tool, this source of evidence enables the detection of indicators that require correlating package-registry information with repository metadata. Unlike local dependency-management indicators, these signals do not emerge solely from the consuming project. Instead, they depend on the consistency between what the package claims in the registry and what can actually be observed in its associated repository. This makes repository metadata especially important for a broader conception of software supply chain trust.

At the same time, repository-based analysis is more complex than purely local analysis. Repository links may be missing, inaccessible, or outdated, and repository platforms may expose metadata in different forms. As a result, this evidence source should be treated as an additional analysis layer that expands transparency-related coverage, rather than as a strict prerequisite for the core functionality of the tool. In practical terms, repository metadata supports a richer supply chain assessment, but it should complement rather than replace the analysis of local files and registry data.

For these reasons, source repository metadata constitutes the third major evidence source considered in this work. It strengthens the ability of the proposed tool to identify provenance- and transparency-related indicators, thereby complementing local dependency inspection and package-registry enrichment with a broader view of dependency origin and traceability.

3.6 Feasibility of the Proposed Approach

The analysis carried out in the previous sections suggests that the conception of the proposed tool is technically feasible within the scope of this project. This feasibility is supported by three main observations: first, the required evidence sources are accessible in practice; second, the literature already defines meaningful categories of supply chain risk indicators; and third, a relevant subset of those indicators can be detected through a lightweight combination of static inspection and selected metadata enrichment [19, 20, 1].

At the local-analysis level, feasibility is particularly strong. Npm projects already expose dependency declarations and resolved dependency information through files such as `package.json` and `package-lock.json`. Since the lockfile describes the exact dependency tree generated during installation, these files provide a practical and reproducible basis for detecting a first layer of indicators without requiring intrusive analysis steps [22, 12]. This makes local dependency-management indicators especially suitable for an initial prototype.

At the metadata-enrichment level, feasibility remains viable, although with additional constraints. Package-registry metadata can be queried in order to inspect package-level information that is not visible in local files alone, and repository-related information can be used to reason about transparency and provenance [23, 1]. This means that the proposed tool can realistically extend its analysis beyond purely local evidence, provided that it treats external metadata as an enrichment layer and not as the only basis for analysis.

To summarize the practical viability of the proposed approach, Table 3.2 presents the three main groups of indicators considered in this work, their primary evidence sources, and their expected feasibility in the first prototype of the tool.

In this work, the term *indicator* refers to a supply chain risk indicator, that is, an observable pattern derived from local project data or external metadata that may reveal a potential maintainability or security concern associated with a third-party dependency.

Table 3.2: Feasibility of indicator groups in the first prototype of the proposed tool.

Indicator group	Main evidence source	Feasibility in first prototype	Main considerations
Local dependency-management indicators	<code>package.json</code> , <code>package-lock.json</code>	High	Based on local and stable project files; suitable for lightweight static analysis.
Registry- and maintainer-related indicators	Package registry metadata	Medium	Feasible through metadata enrichment, but dependent on external availability and metadata quality.
Provenance and repository-related indicators	Source repository metadata and cross-checks with registry data	Medium to Low	More sensitive to missing links, inaccessible repositories, and consistency issues across sources.

As shown in Table 3.2, local dependency-management indicators are the most suitable starting point for the first prototype, since they rely on stable project files that are immediately available for analysis. By contrast, registry-related and repository-related indicators remain feasible, but their implementation depends more strongly on external metadata enrichment and on the consistency of information across sources.

The feasibility of the approach also depends on the fact that the proposed tool does not attempt to solve the entire supply chain security problem. Instead, it targets a bounded and realistic objective: the detection and reporting of warning signs derived from observable project and ecosystem evidence. This narrower objective is important because it allows the tool to remain lightweight and developer-oriented, while still producing useful findings that complement conventional vulnerability-oriented workflows [13, 14, 1].

At the same time, feasibility does not imply completeness. Some indicators are easier to detect than others, and some may require data that is missing, inconsistent, or temporarily unavailable. Furthermore, the exact dependency state of a project may still depend on contextual installation conditions. For example, npm documents that flags used during installation may affect the shape of the dependency tree and may need to be reproduced consistently in later executions [24]. Therefore, the proposed approach should be understood as feasible for a meaningful subset of indicators, rather than as a complete or definitive solution to all supply chain concerns.

Overall, the proposed approach is feasible because it relies on evidence sources and analysis steps that are already available in the npm ecosystem, while restricting its ambition to a realistic developer-support problem. This establishes a sound basis for the design of the proposed tool, which is presented in the following chapter.

3.7 Use Case View

The use case view describes the interaction between the developer and the proposed tool at a functional level. Since the solution is implemented as a developer-oriented command-line tool, the primary actor is the *Developer*, who provides an npm project, selects or configures the analysis, executes the tool, and inspects the generated results. Figure 3.1 presents the use case diagram of the proposed solution. The central use case is *Analyze npm project*, which represents the main goal of the system. This use case includes the configuration of the analysis, the selection of smells, the execution of local smell detection, and the generation of output artifacts.

After the report is generated, the developer can inspect the detected findings and the recorded execution errors. The diagram also represents optional analysis extensions. Knip-based detection is used when dependency usage analysis is required,

namely for unused and missing dependencies. Dirty-Waters-based detection is used when ecosystem and provenance-related smells are selected.

Since these external integrations may depend on tool availability, external meta-data, credentials, or execution backends, the diagram also includes the handling of external-tool failures as an extension of those analysis modes. This representation is adequate for the current scope of the work because it keeps the main workflow centered on npm project analysis while still making explicit the most relevant functional variations supported by the prototype: configurable execution, selective smell detection, local analysis, external-tool-based analysis, report generation, and finding inspection.

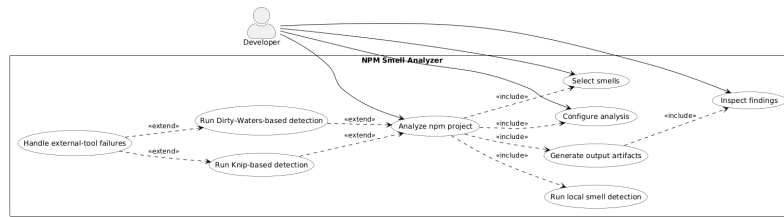


Figure 3.1: Use case diagram of the proposed solution.

As shown in Figure 3.1, the developer interacts with the system through a single main goal: requesting the analysis of a project. The internal steps required to perform this analysis are described more precisely in the following subsection and in the behavioral view presented later in this chapter.

3.7.1 Use Case Description: Analyze Project

The main use case of the proposed solution is *Analyze Project*. In this use case, the developer requests the analysis of an npm project in order to detect a predefined subset of supply chain risk indicators and obtain a structured report describing the detected findings.

Primary actor: Developer.

Goal: Analyze an npm project and report detected supply chain risk indicators associated with its dependencies.

Preconditions: The project is accessible to the tool and the dependency-related files required for analysis are available.

Postconditions: The system produces an analysis result and generates a structured report containing the detected indicators, the affected dependencies, and the available supporting evidence.

Main success scenario:

1. The developer selects the project to be analyzed.

2. The developer chooses the analysis mode, either full analysis or selected smells only.
3. The system reads the relevant project files.
4. The system reconstructs the dependency information required for the analysis.
5. The system executes the applicable local detection rules.
6. When required, the system invokes complementary external tools or metadata queries.
7. The system aggregates the findings obtained from the different analysis sources.
8. The system generates and presents the final analysis report.

Alternative or exceptional flows:

- If external metadata sources or complementary tools are unavailable, the system continues the analysis with partial results and records the corresponding errors or warnings in the final report.
- If the developer selects only a subset of smells, the system executes only the relevant detection rules and supporting analysis mechanisms.

3.8 Requirements and Quality Attributes

The requirements define the main capabilities that the proposed tool shall provide to support the detection and reporting of software supply chain smells in npm projects. Since the tool is intended to be used as a developer-oriented command-line application, these requirements are expressed in terms of the analysis operations, configuration mechanisms, detection capabilities, and output artifacts that the system must provide.

- **FR1. Analyze npm projects:** The tool shall analyze an npm-based project provided by the user through the command-line interface.
- **FR2. Resolve analysis configuration:** The tool shall resolve the analysis configuration from command-line arguments and, when available, from a project-level configuration file.
- **FR3. Select smells for analysis:** The tool shall allow the user to execute all supported smells or a selected subset of smells.
- **FR4. Inspect dependency manifests:** The tool shall inspect `package.json` to extract declared dependencies, development dependencies, and version requirements required by manifest-based detectors.

- **FR5. Inspect dependency lockfiles:** The tool shall inspect `package-lock.json`, when available, to analyze resolved dependency information required by lockfile-based detectors.
- **FR6. Detect local dependency smells:** The tool shall detect local dependency-management smells, including missing lockfiles, pinned dependencies, URL dependencies, restrictive or permissive constraints, duplicate resolved versions, installation scripts, and unmaintained packages.
- **FR7. Detect usage-consistency smells:** The tool shall integrate Knip-based analysis to detect dependency usage inconsistencies, including unused dependencies and missing dependencies.
- **FR8. Detect ecosystem and provenance smells:** The tool shall integrate Dirty-Waters-based analysis to detect ecosystem and provenance-related smells, including deprecated dependencies, missing source-code links, missing source-code SHA or release tags, fork-based dependencies, missing build attestations, invalid code signatures, and aliased packages.
- **FR9. Normalize detected findings:** The tool shall represent detected smells using a unified finding structure containing the smell identifier, affected dependency when applicable, severity, evidence, and explanatory message.
- **FR10. Generate persistent output artifacts:** The tool shall generate persistent output artifacts for each analysis run, including structured JSON files and a human-readable text report.
- **FR11. Report execution errors:** The tool shall record analysis errors and external-tool execution failures without preventing the generation of partial analysis results.
- **FR12. Explain detected findings:** The tool shall provide developer-oriented explanations for detected findings through concise messages that describe the detected smell, the affected dependency when applicable, and the evidence supporting the finding.

In addition to its functional capabilities, the proposed solution must satisfy a set of requirements that constrain how the tool should operate and how its quality should be assessed. Since the proposed system is a developer-oriented analysis tool, these requirements are particularly important to ensure that the solution is not only capable of detecting supply chain risk indicators, but also usable, efficient, robust, secure, reproducible, and maintainable in practice.

These requirements are defined in accordance with the quality-oriented perspective of ISO/IEC 25010, which emphasizes characteristics such as usability, performance efficiency, reliability, security, and maintainability [25]. In the context of the

present work, these quality attributes are interpreted with regard to the specific nature of the solution, namely a tool that performs static and metadata-based analysis of npm project dependencies and produces structured developer-oriented reports.

- **NFR1. Usability:** The tool shall present its results in a structured and understandable form, having software developers as its target audience. Each finding shall include the fields `smell`, `dependency`, `severity`, `evidence`, and `message`. The textual report (`report.txt`) shall summarize the total number of findings and errors and shall list each finding with the affected dependency in a concise single-line description. This requirement is considered satisfied if all generated findings contain the mandatory fields and if the `message` field of each finding is expressed as a natural-language sentence with no more than 180 characters.
- **NFR2. Performance efficiency:** The local native analysis, covering detectors that do not depend on external tools, shall complete in less than one second for npm projects with up to 50 direct dependencies declared in `package.json`, measured on the development machine with Python 3.10 or later. The complete analysis mode including Knip shall complete in less than 30 seconds for the same type of project. Dirty-Waters-based analysis is not included in this threshold because it depends on external repository metadata, GitHub API access, backend configuration, and environment-specific execution conditions.
- **NFR3. Fault tolerance and partial-result reporting:** The tool shall produce a valid report even when one or more external tools, such as Knip or Dirty-Waters, are unavailable or return an unexpected exit code. In such cases, execution errors shall be recorded in `errors.json` and referenced in `report.txt`, while the remaining detectors shall continue to execute whenever possible. This requirement is considered satisfied if, for external-tool failure scenarios covered by mock-based tests, the tool still generates `findings.json`, `errors.json`, and `report.txt` without raising an unhandled exception.
- **NFR4. Reproducibility:** When executed over the same project state, with the same `package.json`, `package-lock.json`, selected smells, and configuration parameters, the tool shall produce identical results in successive executions. This requirement is considered satisfied if three consecutive executions over the same prepared test project produce bit-by-bit identical `findings.json` files for native and Knip-based analyses.
- **NFR5. Security:** The tool shall not execute code from the dependencies being analyzed. Native detectors shall operate exclusively through static inspection of local files and metadata. Subprocess execution shall be restricted to external-tool adapter modules, namely the Knip and Dirty-Waters adapters,

and shall not appear inside individual detector modules under `src/smells/`. In addition, detector modules shall not contain calls to `exec()`, `eval()`, or dynamic import mechanisms that could execute dependency code during analysis. This requirement is verified through static inspection of the source code.

- **NFR6. Maintainability:** The internal design of the tool shall support the addition of new smells, data sources, and reporting rules with limited impact on existing modules. This requirement is evaluated using both architectural evidence and static code metrics. The architectural criterion is that adding a new native detector should require the creation of a new detector module under `src/smells/` and the registration of the corresponding smell in `src/smells/registry.py`, without requiring changes to the orchestrator or CLI execution pipeline. The metric-based evaluation is performed with Radon over the `src/` directory, using the following acceptance criteria: average cyclomatic complexity below 10, fewer than 15% of functions or methods classified with grade C or worse, and Maintainability Index grade A for all analyzed modules. Additionally, individual detector modules shall have low efferent coupling, with no more than three internal imports per detector file, excluding shared registry or common helper modules.
- **NFR7. Low setup complexity:** The tool shall be executable in its default local analysis mode using a standard command-line invocation composed of the target project path and an explicit analysis mode, such as `-all`, `-smell`, or an equivalent reusable configuration file. Installation shall follow a standard Python workflow using `pip install -e .` with Python 3.10 or later. The repository shall also include example configuration files covering the most common execution modes. This requirement is considered satisfied if the tool can be installed in a clean Python environment, if the command-line entry point is available after installation, if the default local analysis can be executed without Dirty-Waters-specific setup, and if the repository includes example configuration files under `config_examples/`.

Among these requirements, some are directly suitable for later quantitative evaluation. In particular, performance efficiency should be assessed through execution-time measurements under controlled project sizes and machine configurations, while fault tolerance may be assessed through the percentage of executions that still generate a valid report when an external dependency or metadata source fails. Likewise, maintainability should be evaluated with architecture-oriented analysis tools and metrics, rather than only through qualitative judgment.

Taken together, these requirements ensure that the proposed solution remains aligned with the main design goals of this work: lightweight operation, understandable reporting, safe dependency analysis, robustness in the presence of incomplete evidence, and future extensibility of the implemented smell-detection mechanisms.

Chapter 4

Solution Conception

This chapter describes the conception of the *NPM Smell Analyzer* based on the requirements previously identified. It starts by outlining the design goals that guide the solution. The chapter then presents a structural view of the system through a refined Domain Model and a behavioral view through sequence diagrams that illustrate the interaction between components. Finally, it details the smell detection strategy, explaining the hybrid approach used to map various risk indicators to native detection rules or external tool adapters, providing the technical basis for the implementation described in the following chapter.

4.1 Design Goals and Requirements Basis

The conception of the proposed solution is grounded on the problem analysis developed in Chapter 3, namely the identified indicator categories, the available evidence sources, the feasibility assessment, and the main functional and non-functional requirements. Rather than designing a general-purpose dependency analysis platform, the objective of the present work is to define a lightweight and developer-oriented tool capable of detecting a feasible subset of supply chain risk indicators in npm projects.

From a functional perspective, the proposed solution must support the analysis of dependency-related project files, the reconstruction of dependency information, the detection of local and metadata-based indicators, the integration of complementary external tools when necessary, and the generation of structured reports. From a

quality perspective, the conception of the solution is guided by goals such as usability, reproducibility, limited setup complexity, tolerance to partial failures in external metadata collection, and future maintainability.

These requirements directly influence the design strategy adopted in this chapter. First, the solution is conceived as a modular analysis workflow, so that different categories of smells can be supported through distinct detection mechanisms. Second, the solution combines local analysis with selective external enrichment, instead of relying exclusively on a single analysis source. Third, the reporting component is treated as part of the core design, since the usefulness of the tool depends not only on detecting indicators, but also on presenting them in a clear and actionable way.

For these reasons, the conception presented in the following sections is organized around three complementary views of the solution: a structural view, expressed through the domain model; a functional interaction view, expressed through the use case view; and a behavioral view, expressed through the sequence of interactions performed during project analysis. Together, these views provide the conceptual basis for the implementation of the proposed tool.

4.2 Domain Model

The domain model of the proposed solution identifies the main concepts involved in the analysis of supply chain risk indicators in npm projects. Its purpose is to provide a structural view of the solution domain, clarifying the entities manipulated by the tool and the relationships between project data, detection rules, external tools, collected evidence, and generated results.

Figure 4.1 presents the domain model adopted for the proposed tool. At the center of the model is the **Project**, which contains the manifest and lockfile information required to identify declared and resolved dependencies. These dependencies constitute the main analysis target of the system, since they may be associated with one or more supply chain risk indicators.

The model also reflects the fact that smell detection is evidence-based. For this reason, the **Finding** entity is associated with **Evidence**, which represents the information collected from local files, metadata sources, or complementary external tools. In turn, **Detector** captures the logic used to determine whether a given indicator is present, while **MetadataSource** and **ExternalTool** represent the origins of additional information required for some categories of smells.

The analysis process itself is represented through the entities **AnalysisRequest**, **AnalysisResult**, and **Report**. This part of the model makes explicit that the proposed solution is not limited to isolated smell checks, but instead supports a complete analysis workflow in which a project is analyzed, findings are consolidated, and a final report is produced for the developer.

Overall, the domain model shows that the proposed solution is structured around four main concerns: project dependency representation, smell detection rules, evidence collection, and report generation. This structural perspective provides the conceptual basis for the interaction and behavioral views presented in the following sections.

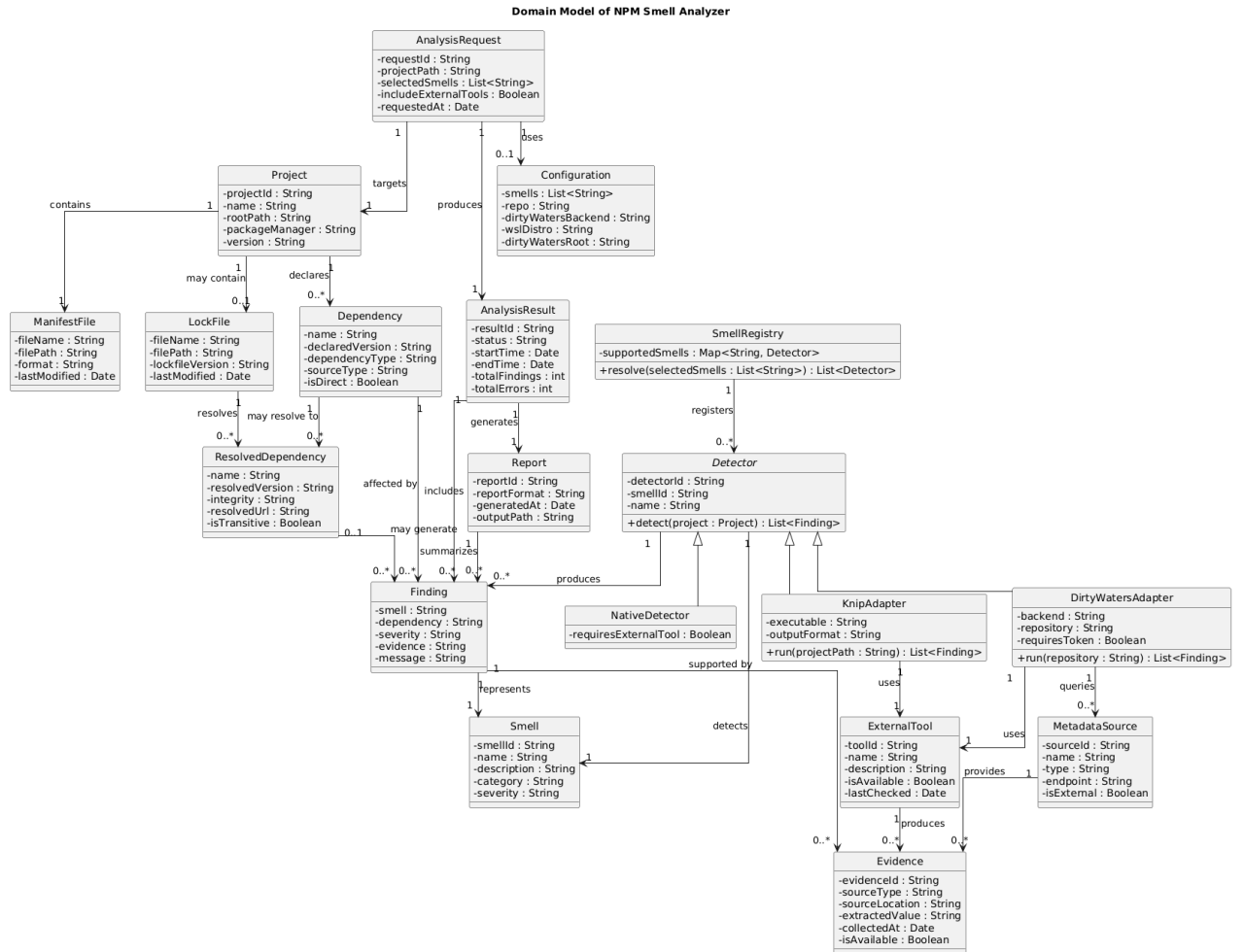


Figure 4.1: Domain model of the proposed supply chain risk analysis tool.

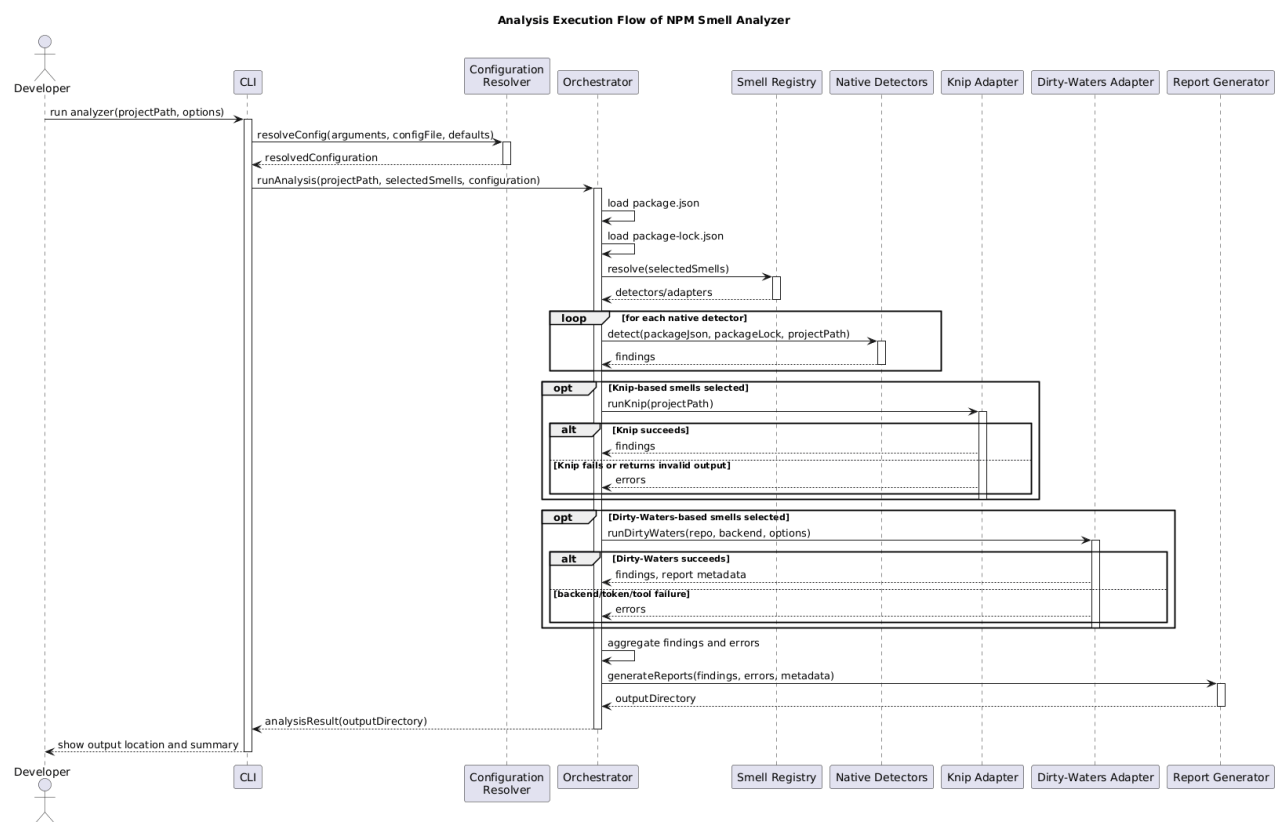
4.3 Behavioral View

While the domain model provides a structural view of the proposed solution and the use case view captures the main user interaction, the behavioral view describes how the main software components collaborate during project analysis. In the present work, this view is expressed through a sequence diagram that represents the interactions performed when a developer requests the analysis of an npm project.

The behavioral view is particularly relevant because the proposed solution is based on a multi-step workflow. The analysis process involves local project inspection, dependency reconstruction, rule-based detection, optional use of complementary external tools, aggregation of findings, and final report generation. The sequence diagram therefore makes explicit how these actions are coordinated within the system.

4.3.1 Analysis Execution Flow with Local and External Smell Detection

Figure 4.2 presents the analysis execution flow associated with the main use case of the proposed tool. The diagram describes the interactions between the developer, the CLI, the configuration resolver, the Orchestrator, the Smell Registry, the native detectors, the Knip and Dirty-Waters adapters, and the report generator.

Figure 4.2: Sequence diagram for the *Analyze Project* use case.

The sequence begins when the developer selects a project and the desired analysis mode through the interface. The request is then forwarded to the Orchestrator, which creates the corresponding analysis request and coordinates the remaining workflow. The Orchestrator resolves the required detectors through the Smell Registry, reconstructs dependency information, and executes the applicable detection rules.

The diagram also makes explicit that the proposed solution supports different analysis modes. In one branch, the system performs a full analysis of all supported smells. In another branch, the system executes only the subset of smells selected by the developer. This distinction is important because the proposed tool is intended to support both broad inspection and more focused smell verification.

When a selected smell depends on external support, the Orchestrator may invoke the corresponding adapter. The sequence diagram also includes the possibility of partial failures, such as tool unavailability or timeout, in which case the system continues the workflow with partial results and records the corresponding errors.

Finally, the result aggregation component consolidates the findings collected from local and external analysis steps, and the report generator produces the final structured report that is presented to the developer. This behavioral view reinforces the idea that the proposed solution is conceived as an orchestrated analysis workflow rather than as a single isolated check.

4.4 Architectural Patterns and Design Rationale

The architecture of the proposed tool was designed to support modularity, extensibility, and separation of responsibilities. Since the tool must detect different types of supply chain smells, each requiring different evidence sources and detection logic, the design avoids a monolithic analyzer and instead organizes the analysis workflow around a set of independent detectors coordinated by an orchestration layer.

The main design decision is the use of a Strategy-oriented structure for smell detection. Each detector encapsulates the detection logic for one smell or smell family and exposes a common detection operation. This allows the orchestration layer to execute detectors uniformly, without depending on the concrete implementation details of each smell. In this sense, native detectors, Knip-based detectors, and Dirty-Waters-based detectors can be treated as interchangeable detection strategies, selected according to the smells requested for a given analysis run. This follows the intent of the Strategy pattern, which enables different algorithms or behaviours to be encapsulated and used interchangeably through a common interface [26].

Detector discovery is supported by a Registry structure. The smell registry maps smell identifiers, such as `no-package-lock`, `url-dependency`, or `deprecated-dependency`, to the corresponding detector instances. This avoids hard-coding detector selection

logic inside the orchestrator and supports the future addition of new smells by registering new detector implementations. This design follows the general intent of the Registry pattern, which provides a known access point for retrieving objects or services needed by other parts of the system [27].

Figure 4.3 presents a focused class-level view of the Strategy and Registry structures used in the proposed tool. The diagram is not intended to represent the complete implementation, but rather to clarify how the orchestrator obtains detector implementations from the registry and executes them through a common detection contract.

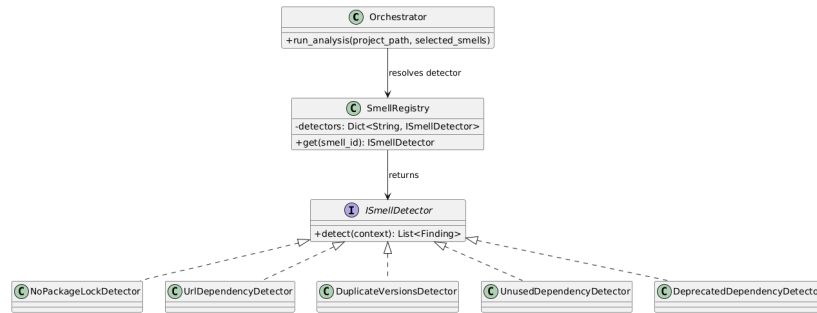


Figure 4.3: Class-level view of the Strategy and Registry structures used for smell detection.

External analysis tools are integrated through Adapter-like modules. The Knip adapter encapsulates the discovery and execution of the Knip command-line tool and converts its JSON output into internal findings. Similarly, the Dirty-Waters adapter encapsulates backend selection, WSL/native execution, GitHub token handling, path conversion, report recovery, and error handling. This follows the intent of the Adapter pattern, since external tools expose execution models and output formats that do not directly match the internal representation used by the proposed tool [26].

Figure 4.4 shows how external tools are isolated behind adapter modules. This design prevents tool-specific execution details, output parsing, backend configuration, and error-handling logic from leaking into the orchestration layer or into the common finding representation.

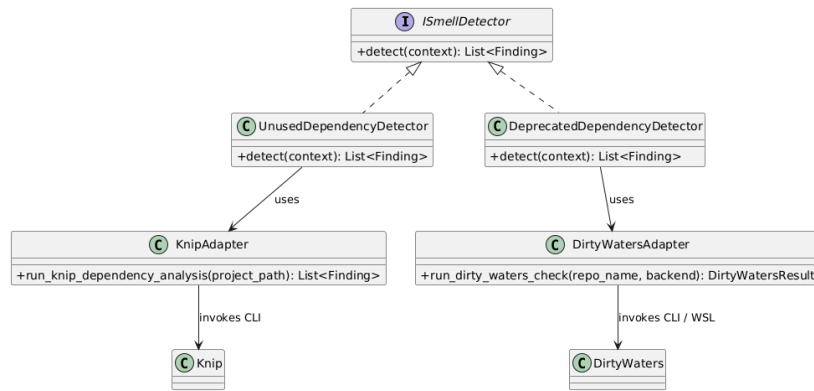


Figure 4.4: Class-level view of the Adapter-based integration of external analysis tools.

Finally, the Orchestrator coordinates the overall workflow. It validates the project path, loads only the evidence required by the selected smells, builds the analysis context, retrieves detectors from the registry, executes them, aggregates findings and errors into the analysis result, and delegates report generation to the report writer. This separation keeps the orchestration logic independent from the concrete detection mechanisms and from the details of external tool execution.

Figure 4.5 presents this organization at component level. The diagram highlights the command-line interface, configuration resolver, orchestration layer, smell registry, detector groups, external-tool adapters, finding model, and report writer. The use of interfaces and assembly connectors emphasizes that the components are connected through explicit contracts rather than direct knowledge of each concrete implementation.

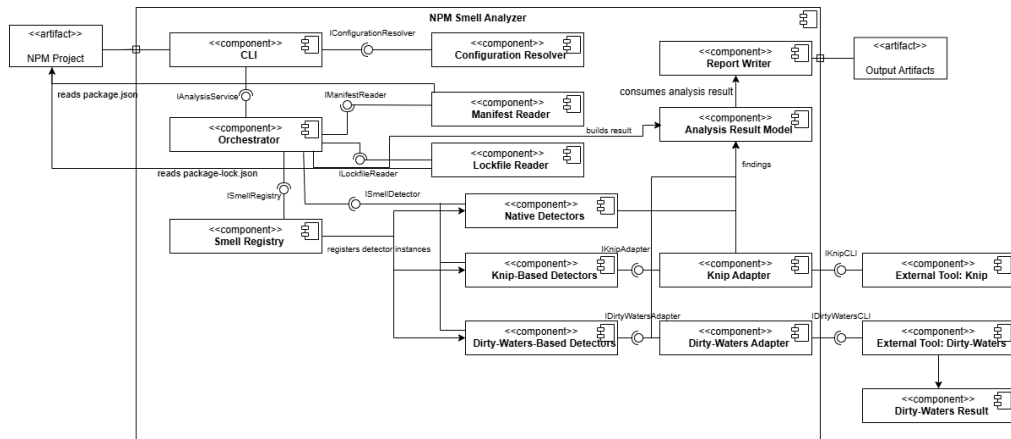


Figure 4.5: Component diagram of the proposed NPM Smell Analyzer architecture.

4.5 Smell Detection Strategy

The smell detection strategy was defined according to the type of evidence required by each smell and according to the feasibility of implementing each detection mechanism within a lightweight developer-oriented prototype. As discussed in the literature review, the relevant indicators considered in this work originate from complementary sources: dependency smells in JavaScript projects [19], weak-link indicators in the npm ecosystem [20], and supply chain smells supported by Dirty-Waters [1, 21]. However, these indicators do not depend on a single evidence source or on a single detection mechanism.

For this reason, different design alternatives were considered before defining the final detection strategy. Table 4.1 compares these alternatives according to their main advantages, limitations, and suitability for the proposed tool.

Table 4.1: Comparison of alternative smell detection strategies considered for the proposed tool.

Alternative	Advantages	Limitations	Decision
Single monolithic analyzer	Simple initial implementation; all detection logic is centralized in a single workflow.	Low extensibility; difficult to add new smells; mixes local analysis, external-tool execution, parsing, and reporting concerns.	Rejected, because it would make the prototype harder to maintain and extend.
Native-only static analysis	Lightweight, fast, reproducible, and independent from external tools.	Cannot detect smells that require dependency usage analysis, repository metadata, provenance information, or ecosystem-level evidence.	Partially adopted for smells based on local files and stable project evidence.
External-tool-only analysis	Reuses existing tools and avoids implementing all detection mechanisms from scratch.	Increases setup complexity; depends on tool availability, external metadata, execution environment, and tool-specific output formats.	Rejected as the only strategy, but adopted selectively through adapters.
Hybrid layered strategy	Combines local static inspection, usage analysis, and ecosystem/provenance checks; supports modular extension through detectors and adapters.	Requires output normalization, error handling, and coordination between heterogeneous evidence sources.	Adopted, because different smells require different evidence sources and detection mechanisms.

Based on this comparison, the proposed tool adopts a hybrid layered detection strategy. The strategy is organized into three main detection layers: native local detectors, Knip-based usage analysis, and Dirty-Waters-based ecosystem/provenance analysis. Table 4.2 summarizes the role of each layer.

Table 4.2: Detection layers adopted in the proposed tool.

Detection layer	Main evidence source	Examples of supported smells	Rationale
Native local detectors	<code>package.json</code> , <code>package-lock.json</code> , installed dependency manifests, and npm registry metadata.	Pinned dependency, URL dependency, restrictive constraint, permissive constraint, no package lock, duplicate versions, installation scripts, and unmaintained package.	Adopted for smells whose evidence can be obtained through lightweight local or metadata-based inspection.
Knip-based analysis	Source usage analysis performed by Knip.	Unused dependency and missing dependency.	Adopted because these smells require usage analysis over the source code, and Knip provides direct support for detecting unused and unlisted dependencies [28].
Dirty-Waters-based analysis	Repository metadata, package metadata, provenance information, and Dirty-Waters reports.	Deprecated dependency, missing source-code link, missing source-code SHA or release tag, dependency on a fork, missing build attestation, invalid code signature, and aliased packages.	Adopted for ecosystem and provenance-related smells that would be costly and fragile to reimplement inside the prototype.

The first layer prioritizes smells with high implementation feasibility and stable local evidence. Manifest-based smells are detected by inspecting dependency declarations in `package.json`, while lockfile-based smells are detected from `package-lock.json`. This choice is consistent with npm documentation, which describes `package-lock.json` as the file that records the exact dependency tree generated during installation and is intended to be committed into source repositories [12]. Additional local checks, such as installation-script detection, are based on npm package scripts and installed dependency manifests [29].

The second layer addresses dependency usage consistency. The original dependency-smell study used *depcheck* for unused and missing dependency detection [19]. However, in the current npm ecosystem, Knip is a more appropriate integration candidate because it explicitly supports unused dependency and missing dependency detection, while Depcheck itself recommends migration to Knip due to maintenance limitations [28, 30]. Therefore, the proposed tool delegates this type of analysis to Knip and converts its output into the internal finding representation.

The third layer addresses smells that require repository-level, package-level, or provenance-related evidence. As previously introduced in Section 2.4.3, Dirty-Waters already implements several checks related to npm supply chain smells. Instead of repeating the detailed description of those checks in this section, the proposed design treats Dirty-Waters as a specialized backend accessed through an adapter. This keeps the core architecture focused on orchestration and normalization, while the external tool remains responsible for complex ecosystem and provenance analysis.

Table 4.3 summarizes the mapping between smell groups, adopted mechanisms, and implementation decisions. This table complements the architectural patterns discussed in Section 4.4, since the selected strategy depends on the combination of detectors, registry-based discovery, adapters, and orchestration.

Table 4.3: Mapping between smell groups, adopted detection mechanisms, and implementation decisions.

Smell group	Adopted mechanism	Implementation status	Justification
Manifest-based dependency smells	Native parsing of <code>package.json</code> .	Implemented.	These smells are directly observable from dependency declarations and version requirements.
Lockfile-based smells	Native parsing of <code>package-lock.json</code> .	Implemented.	Resolved dependency information is available locally and can be inspected without external services.
Dependency usage smells	Knip adapter.	Implemented.	Source usage analysis is delegated to Knip to avoid implementing a complete usage-analysis engine.
Installed dependency metadata smells	Native inspection of installed dependency manifests.	Implemented.	Installation scripts can be detected from dependency-level <code>package.json</code> files when <code>node_modules</code> is available.
Registry-based maintenance smells	Native metadata heuristic.	Implemented.	Package maintenance can be approximated using configurable inactivity thresholds over npm registry metadata.
Repository and provenance smells	Dirty-Waters adapter.	Implemented.	These smells require external repository, package, and provenance evidence that Dirty-Waters already supports.
Maintainer ecosystem smells	Custom ecosystem-wide metadata analysis.	Not implemented.	Signals such as overloaded maintainer, too many maintainers, too many contributors, and expired maintainer domain require broader maintainer-level metadata and additional validation.

This strategy provides a balance between feasibility, extensibility, and evidence coverage. Local rules preserve the lightweight nature of the tool and provide deterministic results for smells that can be detected from project files. External tools are integrated only when their analysis capabilities provide evidence that would be difficult to reproduce reliably within the prototype. Finally, smells requiring broader ecosystem-wide maintainer analysis are left outside the implemented scope, since they depend on metadata sources and validation criteria that exceed the objectives of the first prototype.

4.5.1 Mapping Between Smells and Detection Mechanisms

Since the three main reference works considered in this project do not define a single homogeneous catalogue of indicators, the design of the proposed solution requires an explicit mapping between each smell (or signal), the evidence needed to observe it, and the mechanism that can be used to detect it. This mapping is important because it supports the architectural decision of conceiving the tool as a hybrid analysis solution, combining native local rules, metadata-based checks, and complementary external tools.

To improve readability, this mapping is presented in two complementary tables. Table 4.4 provides a compact overview of the supply chain smells considered in this work, their origin in the literature, the main evidence source associated with each one, and their current support status in the prototype. Table 4.5 then presents the corresponding detection mechanisms and implementation decisions adopted for each smell.

As shown in Table 4.4, the smells associated with dependency declarations and lockfile structure are generally the most suitable for direct implementation through local static analysis. In contrast, several repository-level supply-chain indicators are better supported through integration with specialized external tools, namely Dirty-Waters, while some weak-link signals remain outside the scope of the current lightweight prototype due to their dependence on richer external ecosystem metadata.

Table 4.4: Overview of the supply chain smells considered in this work and their support status in the prototype.

Smell / indicator	Origin	Main evidence source	Prototype support
Pinned dependency	Dependency Smells	<code>package.json</code>	Yes (native)
URL dependency	Dependency Smells	<code>package.json</code>	Yes (native)
Restrictive constraint	Dependency Smells	<code>package.json</code>	Yes (native)
Permissive constraint	Dependency Smells	<code>package.json</code>	Yes (native)
No package lock	Dependency Smells	Project files	Yes (native)
Multiple versions of the same package	Dependency Smells	<code>package-lock.json</code>	Yes (native)
Unused dependency	Dependency Smells	Source usage analysis	Yes (Knip)
Missing dependency	Dependency Smells	Source usage analysis	Yes (Knip)
Deprecated dependency	Weak Links	GitHub/npm metadata	Yes (Dirty-Waters via WSL)
No source code repository link	Weak Links	GitHub/package metadata	Yes (Dirty-Waters via WSL)
No source code SHA / release tag	Weak Links	GitHub/package metadata	Yes (Dirty-Waters via WSL)
Depends on a fork	Weak Links	GitHub metadata	Yes (Dirty-Waters via WSL)
No build attestation	Weak Links	Registry / provenance metadata	Yes (Dirty-Waters via WSL)
No / invalid code signature	Weak Links	Registry / package metadata	Yes (Dirty-Waters via WSL)
Aliased packages	Weak Links	Package manager metadata	Yes (Dirty-Waters via WSL)
Installation scripts	Weak Links	Installed dependency metadata	Yes (native, requires <code>node_modules</code>)

Unmaintained package	Weak Links	npm registry metadata	Yes (native heuristic)
Overloaded maintainer	Weak Links	Maintainer ecosystem meta- data	No (outside prototype scope)
Too many contributors	Weak Links	Repository contributor meta- data	No (outside prototype scope)
Too many maintainers	Weak Links	Registry maintainer metadata	No (outside prototype scope)
Expired maintainer domain	Weak Links	Maintainer email/domain metadata	No (outside prototype scope)

Table 4.5 complements this overview by showing the concrete detection strategy associated with each smell, including whether it was implemented natively, delegated to an external tool, approximated through heuristic analysis, or excluded from the scope of the current prototype.

Table 4.5: Detection mechanisms and implementation decisions adopted for the supply chain smells considered in the prototype.

Smell / indicator	Detection mechanism / candidate tool	Implementation status	Notes
Pinned dependency	Native manifest parsing with exact-version rule validation	Implemented	Fully local rule over dependency declarations in <code>package.json</code> .
URL dependency	Native manifest parsing with dependency source inspection	Implemented	Detects URL-, Git-, or repository-based dependency declarations directly from <code>package.json</code> .
Restrictive constraint	Native manifest parsing with semver policy validation	Implemented	Based on predefined rules for restrictive declarations such as tilde ranges.
Permissive constraint	Native manifest parsing with semver policy validation	Implemented	Based on predefined rules for permissive declarations such as wildcard or open-ended ranges.
No package lock	Native project file inspection	Implemented	Checks whether <code>package-lock.json</code> is present.
Multiple versions of the same package	Native lockfile parsing	Implemented	Detects packages resolved to multiple versions in <code>package-lock.json</code> .
Unused dependency	Knip integration	Implemented	Delegated to Knip for source usage analysis.
Missing dependency	Knip integration	Implemented	Delegated to Knip for used-but-unlisted dependency detection.
Deprecated dependency	Dirty-Waters integration via WSL	Implemented	Executed through Dirty-Waters; requires GitHub repository metadata and WSL/Ubuntu support.
No source code repository link	Dirty-Waters integration via WSL	Implemented	Executed as an external repository-level check.
No source code SHA / release tag	Dirty-Waters integration via WSL	Implemented	Executed with Dirty-Waters checks for source repository and SHA/tag accessibility.

Continued on next page

Table 4.5: Detection mechanisms and implementation decisions adopted for the supply chain smells considered in the prototype.
(continued)

Smell / indicator	Detection mechanism / candidate tool	Implementation status	Notes
Depends on a fork	Dirty-Waters integration via WSL	Implemented	Executed as a repository-level metadata check through Dirty-Waters.
No build attestation	Dirty-Waters integration via WSL	Implemented	Based on provenance / attestation metadata collected by Dirty-Waters.
No / invalid code signature	Dirty-Waters integration via WSL	Implemented	Based on code-signature-related metadata collected by Dirty-Waters.
Aliased packages	Dirty-Waters integration via WSL	Implemented	Detected through Dirty-Waters package-manager analysis.
Installation scripts	Native inspection of installed dependency manifests	Implemented	Detects <code>preinstall</code> , <code>install</code> , and <code>postinstall</code> scripts in dependency <code>package.json</code> files; requires locally installed <code>node_modules</code> .
Unmaintained package	Native heuristic using npm registry metadata	Implemented	Prototype heuristic based on the elapsed time since the last registry modification.
Overloaded maintainer	Maintainer-level cross-package analysis	Not implemented	Outside the scope of a lightweight local prototype because it depends on external ecosystem-wide maintainer metadata.
Too many contributors	Repository contributor analysis	Not implemented	Requires contributor metadata aggregation and threshold calibration; excluded from prototype scope.

Continued on next page

Table 4.5: Detection mechanisms and implementation decisions adopted for the supply chain smells considered in the prototype.
(continued)

Smell / indicator	Detection mechanism / candidate tool	Implementation status	Notes
Too many maintainers	Registry maintainer analysis	Not implemented	Requires reliable maintainer metadata from external sources and policy calibration.
Expired maintainer domain	Domain / email validation over maintainer metadata	Not implemented	Considered too fragile for the current prototype due to dependence on external domain-validation checks and higher false-positive risk.

As shown in Table 4.5, the final prototype combines three complementary strategies: native local analysis for manifest- and lockfile-based smells, Knip-based detection for dependency usage inconsistencies, and Dirty-Waters integration for repository-level supply chain indicators. The remaining weak-link signals were intentionally left outside the scope of the prototype, since their detection would require richer external maintainer or ecosystem metadata and would introduce additional complexity and a higher risk of false positives.

4.6 Design Constraints and Trade-offs

The conception of the proposed tool is subject to a set of practical constraints and design trade-offs that directly influence its scope, implementation strategy, and analytical capabilities. These aspects must be made explicit because the goal of the project is not to maximize coverage at any cost, but to define a feasible and useful approach for detecting supply chain risk indicators in npm projects.

A first constraint concerns the availability and quality of evidence sources. Local files such as `package.json` and `package-lock.json` provide a stable and reproducible basis for analysis, but some indicators require external metadata obtained from the package registry or from the source repository. As a consequence, the overall completeness of the analysis may depend on the availability, consistency, and freshness of external sources [12, 20, 1]. This creates a trade-off between analytical coverage and operational simplicity: the more the tool depends on external enrichment, the richer the possible analysis becomes, but the more it may be affected by missing metadata, unavailable services, or source inconsistencies.

A second constraint concerns the distinction between lightweight static analysis and deeper forms of validation. The present work deliberately favors static inspection of local files and metadata sources, since this supports a safer and more lightweight workflow. However, this also means that the tool is designed to detect indicators and warning signs rather than to confirm exploitability, prove malicious intent, or validate runtime behavior. This leads to an important trade-off between precision and practicality: a lightweight approach is easier to integrate into developer workflows, but it may also produce findings that require manual interpretation or follow-up investigation.

A third constraint concerns reproducibility and dependency resolution. In npm-based projects, the lockfile provides an important reference for the resolved dependency tree, but the exact dependency state may still be influenced by project configuration, installation conditions, or the absence of complete lockfile information [12, 18]. Therefore, there is a trade-off between relying on deterministic local inputs and acknowledging that some analysis results are still contextual. This is

particularly relevant when the tool is expected to operate consistently across local environments and continuous integration pipelines.

A fourth constraint concerns the heterogeneity of the indicator catalogue itself. The indicators considered in this work originate from different strands of literature and do not all have the same nature, the same data requirements, or the same degree of interpretability [19, 20, 1]. Some are directly observable from local dependency declarations, while others depend on more indirect evidence involving maintainers, provenance, or repository transparency. This creates a trade-off between conceptual breadth and implementation focus: including more categories may increase the relevance of the tool, but it may also increase complexity and reduce clarity if the detected findings are not presented carefully.

A final trade-off concerns reporting. A purely technical output may maximize detail, but it may also reduce usability for the intended audience. Conversely, a highly simplified report may be easy to read, but may omit relevant context required for informed decision-making. For this reason, the proposed tool must balance analytical richness with clarity of presentation, so that findings remain both understandable and actionable for developers.

In conclusion, these constraints and trade-offs reinforce the need for a balanced conception strategy. The proposed tool should aim to provide meaningful and well-explained supply chain risk indicators while remaining lightweight, reproducible, and practical enough for developer-oriented use.

Chapter 5

Solution Implementation

This chapter presents the implementation of the proposed prototype, named *Npm Smell Analyzer*. After defining the conceptual solution and its detection strategy in the previous chapter, the focus now shifts to how that solution was concretely realized as a working command-line tool for npm projects. The chapter describes the main architectural components of the implementation, the adopted detection mechanisms for native, Knip-based, and Dirty-Waters-based smells, and the main design decisions taken during development. It also discusses the configuration mechanism introduced to improve usability, the supporting artifacts included in the repository, and the validation process carried out through unit, integration, and mock-based tests. In this way, the chapter provides a practical view of how the proposed solution was translated into an executable and reproducible prototype, whose source code is publicly available as an open-source contribution [31].

5.1 Overview of the Implemented Prototype

The proposed solution was implemented as a command-line prototype named *NPM Smell Analyzer* [31]. Its main objective is to support the detection of selected software supply chain smells in npm-based projects by combining lightweight local analysis with the integration of specialized external tools. Rather than focusing on a single type of indicator, the prototype was designed to support multiple complementary detection strategies within a unified execution flow.

At a functional level, the prototype analyzes project artifacts such as `package.json`, `package-lock.json`, and, in selected cases, installed dependency metadata from `node_modules`. In addition, it integrates Knip to identify dependency-usage inconsistencies and Dirty-Waters to obtain repository-level supply chain indicators that cannot be inferred from local metadata alone. This hybrid design allows the prototype to cover a broader set of smells than a purely local analysis approach.

The implementation was organized to remain lightweight, modular, and reproducible. Native detectors, external-tool adapters, configuration loading, orchestration, and report generation were implemented as separate concerns, making the prototype easier to maintain, test, and extend. This modularity was particularly important given the need to support both fast local analysis and more environment-dependent external integrations.

From the user perspective, the prototype is executed through a CLI and produces structured output artifacts, namely `findings.json`, `errors.json`, and `report.txt`. It also supports configuration files that simplify recurring analyses and reduce the need to pass long command-line invocations, especially for Dirty-Waters-based checks.

The remainder of this chapter explains how this prototype was implemented in practice. It presents the implementation architecture, the adopted detection mechanisms, the integration of external tools, the configuration design, the main development decisions and trade-offs, and the testing strategy used to validate the implemented solution.

5.2 Implementation Architecture

The implemented prototype, named *NPM Smell Analyzer*, was designed as a command-line tool organized around a modular analysis pipeline. The main goal of the implementation was to provide a lightweight and extensible structure capable of supporting different categories of software supply chain smells while keeping the execution flow understandable and easy to validate.

At a high level, the architecture is divided into five main responsibilities: command-line interaction, configuration loading, orchestration of the analysis process, smell detection, and report generation. This separation was adopted to reduce coupling between components and to simplify both maintenance and testing.

The execution starts in the command-line interface, implemented in `cli.py`. This component is responsible for parsing user arguments, loading configuration values when available, resolving the effective execution mode, and validating the selected smells. It also enforces precedence rules between command-line arguments and configuration files, ensuring that explicit user input overrides default or preconfigured values.

After the initial execution context is resolved, control is passed to the orchestration layer, implemented in `orchestrator.py`. This component coordinates the analysis as a whole. It determines which project files need to be read according to the selected smells, prepares the shared analysis context, invokes the corresponding smell detectors, collects findings and errors, and finally forwards the resulting analysis object to the report writer. The orchestrator therefore acts as the central coordination point of the prototype.

The smell detectors themselves are organized as independent analysis units registered in a common smell registry. This design makes it possible to add, remove, or refine smells without changing the overall execution pipeline. Native smells are implemented directly in Python and operate over local project artifacts such as `package.json`, `package-lock.json`, or installed dependency metadata. In contrast, smells that depend on external analysis tools are delegated to dedicated adapters, which encapsulate the interaction with those tools and convert their outputs into the internal `Finding` representation used throughout the prototype.

Two external integrations were implemented. The first is Knip, used to detect dependency-usage inconsistencies such as unused and missing dependencies. The second is Dirty-Waters, used to extend the prototype with repository-level supply chain checks that cannot be derived from local npm metadata alone. In order to avoid spreading subprocess and environment-specific logic across the codebase, both tools are isolated behind adapter modules. This was an important implementation decision, since it allowed the prototype to keep a uniform internal interface even when relying on heterogeneous external tools.

The final stage of the pipeline is report generation. After the execution of all selected detectors, the findings and errors are persisted into structured output artifacts. These include machine-readable JSON files as well as a human-readable text report, making the results suitable both for direct inspection and for later automation or experimentation.

Overall, the implemented architecture reflects the hybrid nature of the prototype. On one hand, it supports fast local static analysis through native detectors. On the other hand, it remains open to specialized external tooling when richer ecosystem or repository metadata is required. This balance between modularity, practicality, and extensibility guided the implementation of the prototype.

5.2.1 CLI and Execution Flow

The execution flow of the prototype starts at the command-line interface, implemented in `cli.py`. This component was designed as the main entry point of the tool and is responsible for transforming user input into a validated analysis request.

At this stage, the prototype parses the command-line arguments provided by the user, including the path of the target npm project, the execution mode, the

selected smells, and optional integration-specific parameters. In order to support both simple and repeatable executions, the CLI was designed to work together with a configuration mechanism. The analyzer can load configuration values from a project-level configuration file or from a file explicitly passed through the `-config` argument. Command-line arguments always take precedence over configuration values, ensuring that explicit user input can override predefined defaults when necessary.

The CLI supports three main execution modes. The first is the default `-all` mode, which runs the local native smells together with the Knip-based smells. The second is the dedicated `-dirty-waters-all` mode, introduced to execute only the Dirty-Waters-based smells. The third mode allows the user to select one or more individual smells explicitly through repeated `-smell` arguments. This separation was intentionally introduced because Dirty-Waters-based checks are slower, require repository-level metadata, and depend on additional environmental conditions that are not necessary for the native and Knip-based analyses.

After parsing the inputs, the CLI resolves the effective smell selection and validates it against the supported smell registry. It also validates execution-specific constraints, such as requiring a GitHub repository identifier when Dirty-Waters-based smells are selected, or ensuring that the unmaintained-package threshold is a positive integer. These validations were added to fail early and provide clearer feedback to the user before the analysis pipeline is executed.

Once the final execution parameters are resolved, the CLI forwards them to the orchestration layer. After the analysis completes, it prints a concise execution summary, including the output directory and the list of executed smells. In this way, the command-line interface acts not only as the user-facing entry point of the prototype, but also as an important validation and configuration resolution layer.

5.2.2 Orchestration Layer

The core coordination logic of the prototype is implemented in `orchestrator.py`. This component is responsible for managing the full analysis workflow after the execution parameters have been resolved by the command-line interface. Its main purpose is to centralize the execution flow while keeping the individual smell detectors independent from one another.

The orchestrator begins by validating the target project path and initializing a shared `AnalysisResult` object, which stores the selected smells, the accumulated findings, and the errors generated during execution. This object serves as the central container for the results of a full analysis run and is later passed to the report-writing stage.

A key responsibility of the orchestration layer is to determine which project artifacts need to be read according to the selected smells. Smells based on dependency declarations require access to `package.json`, while smells related to resolved

dependency versions require access to `package-lock.json`. Instead of loading all files unconditionally, the orchestrator only reads the files that are actually needed for the selected analysis mode. This reduces unnecessary processing and keeps the execution pipeline more efficient.

Once the relevant inputs are loaded, the orchestrator prepares a shared execution context containing the project path, parsed manifest data, parsed lockfile data, Dirty-Waters-related parameters, and additional smell-specific settings such as the threshold used for the unmaintained-package heuristic. This context is then passed to each selected detector. In practice, this means that all smell detectors follow the same invocation pattern, regardless of whether they implement native logic or depend on an external tool integration.

The orchestrator uses a registry-based approach to resolve the detector associated with each smell. For every selected smell, it retrieves the corresponding detector from the common smell registry and invokes its `detect(...)` method. The resulting findings are appended to the shared result object, while execution failures are captured and recorded as errors instead of interrupting the whole analysis. This design decision was important to preserve robustness: if one smell fails, the remaining ones can still be executed and reported.

Another relevant implementation detail is that the orchestration layer also defines the smell groupings used throughout the prototype. In particular, it distinguishes Dirty-Waters-based smells from the remaining smells in order to support the separate execution mode introduced in the CLI. It also groups smells according to the project artifacts they require, which simplifies conditional input loading.

Finally, after all selected detectors have been executed, the orchestrator passes the populated `AnalysisResult` object to the report writer. The report writer then generates the corresponding output artifacts for the execution. As such, the orchestration layer acts as the central coordination component of the implemented prototype, linking user input, artifact loading, smell execution, and result persistence into a single coherent workflow.

5.2.3 Smell Detectors

The smell detectors constitute the core analysis units of the prototype. Each detector is responsible for implementing the logic associated with one smell and for returning its results in a common internal format. This design was adopted to keep the detection logic modular and to make the prototype easier to extend as new smells are added.

All detectors follow the same execution model: they receive a shared context prepared by the orchestration layer and return a list of `Finding` objects. This context may include the project path, parsed manifest data, parsed lockfile data, repository information, or additional configuration values. By standardizing the

detector interface, the prototype avoids the need for smell-specific orchestration logic and keeps the execution flow uniform across different categories of smells.

The implemented detectors can be divided into three groups. The first group corresponds to native local smells, implemented directly in Python. These detectors rely on local project artifacts such as `package.json`, `package-lock.json`, or installed dependency metadata under `node_modules`. Examples include pinned dependencies, URL-based dependencies, restrictive or permissive version constraints, duplicate resolved versions, installation lifecycle scripts, and the unmaintained-package heuristic.

The second group corresponds to Knip-based smells. In these cases, the detector itself does not implement the full analysis logic directly. Instead, it delegates the analysis to the Knip adapter, which runs Knip and converts its output into the internal `Finding` representation. This approach was adopted for smells such as unused and missing dependencies, where an established specialized tool already provides more reliable source-usage analysis than a lightweight reimplementing would.

The third group corresponds to Dirty-Waters-based smells. These detectors were implemented as thin wrappers over a common Dirty-Waters helper. Their role is to define the smell-specific configuration, invoke the Dirty-Waters adapter with the correct parameters, and interpret the resulting report. This design reduces duplication, since several of these smells share the same external execution mechanism and differ mainly in the check flags and summary patterns used to interpret Dirty-Waters reports.

A registry-based organization was used to make these detectors accessible to the orchestrator. Each smell is associated with a detector instance in a central registry, allowing the prototype to resolve and execute smells dynamically from their names. This registry-based approach also supports the separation between local/Knip-based smells and Dirty-Waters-based smells, which is reflected both in the execution modes exposed by the CLI and in the implementation structure itself.

Overall, the smell detector design reflects the main implementation principle of the prototype: to combine heterogeneous analysis strategies behind a single uniform internal interface. This made the solution easier to test, easier to document, and easier to evolve during the implementation process.

5.2.4 External Tool Adapters

A central implementation decision of the prototype was to encapsulate external tool integrations behind dedicated adapter modules. This decision was motivated by the need to isolate subprocess execution, environment-specific behavior, and output parsing from the rest of the codebase. Without such an abstraction, the orchestration logic and the smell detectors would become tightly coupled to external command execution details, reducing maintainability and making testing more difficult.

Two adapters were implemented: one for Knip and one for Dirty-Waters. Although both integrations rely on external tools, they play different roles in the prototype and therefore required different implementation strategies.

The Knip adapter is responsible for locating a valid Knip command in the target environment, invoking it with the appropriate arguments, and parsing its JSON output. This output is then transformed into **Finding** objects representing unused or missing dependencies. The adapter also handles error scenarios such as unavailable commands, invalid output, unexpected return codes, and execution failures. Since the objective of the adapter is to normalize Knip results into the internal prototype model, the rest of the system can treat Knip-based smells in the same way as native smells.

The Dirty-Waters adapter is more complex due to the characteristics of the external tool and the execution environment required in the project context. Dirty-Waters was integrated to support repository-level software supply chain checks, but in the tested environment it required special handling, particularly on Windows, where the adopted execution path relied on WSL/Ubuntu. For this reason, the adapter was designed not only to invoke Dirty-Waters, but also to manage backend selection, token propagation, debug artifact persistence, report path extraction, and recovery of the generated analysis report. In addition to a disabled backend, the adapter supports explicit WSL-based execution, native execution, and an automatic mode that selects the most appropriate backend according to the operating system.

Another important aspect of the adapter design is that both integrations normalize heterogeneous external outputs into a consistent internal format. Rather than exposing raw subprocess results to the detectors or the orchestrator, the adapters return findings or structured execution results together with explicit error information. This made it possible to validate external-tool integration through mock-based tests, without depending on the real presence of Knip or Dirty-Waters in every test scenario.

By isolating these integrations behind adapters, the prototype achieves a cleaner separation of concerns. The smell detectors remain focused on smell semantics, the orchestration layer remains focused on coordination, and the environment-specific subprocess logic is restricted to well-defined integration points. This was an important design choice for keeping the implementation robust and testable.

5.2.5 Report Generation

The final stage of the implemented analysis pipeline is report generation. After all selected detectors have been executed, the resulting **AnalysisResult** object is passed to the report writer, which is responsible for persisting the outcome of the execution into structured artifacts.

The prototype generates three main output files for each run: `findings.json`, `errors.json`, and `report.txt`. The JSON artifacts provide machine-readable representations of the detected smells and execution errors, making them suitable for automated inspection or later experimentation. The text report provides a concise human-readable summary of the executed smells, the total number of findings, and the recorded errors. This combination of machine-readable and human-readable outputs was intentionally adopted to support both developer-oriented use and academic evaluation of the prototype.

The report generation step also contributes to the reproducibility of the implementation. Instead of keeping results only in memory or only printing them to the terminal, the prototype persists them in an organized output directory, allowing the results of different executions to be inspected, compared, and documented. This proved particularly useful during testing and during the manual validation of the Dirty-Waters integration, where preserving intermediate execution artifacts was important for understanding integration behavior.

Overall, the reporting mechanism complements the modular architecture of the prototype by ensuring that all analysis results are persisted in a consistent and inspectable form.

5.3 Implementation Details

After presenting the overall architecture of the prototype, this section focuses on the most relevant implementation details. Since the full codebase is relatively extensive, the goal is not to reproduce every implementation element, but rather to highlight representative excerpts that illustrate the main design decisions adopted in the development of *NPM Smell Analyzer*.

The selected excerpts emphasize six aspects of the implementation: command-line and configuration resolution, registry-based smell dispatch, the shared structure adopted Dirty-Waters-based smells, Knip integration, Dirty-Waters integration, and reusable configuration profiles. These fragments were chosen because they represent the core execution flow of the prototype and clearly illustrate how the implemented solution combines local static analysis with external tool integration.

5.3.1 CLI and Configuration Resolution

One of the most important implementation decisions taken during development was the introduction of a configuration mechanism combined with explicit command-line overrides. Instead of requiring the user to provide all parameters in every execution, the prototype can load configuration values from a project-level configuration file or from an explicitly selected configuration file. At the same time, command-line arguments always take precedence over configuration values.

This logic is implemented in the CLI layer, which loads the configuration, resolves the effective smell set, validates mandatory constraints, and forwards the final execution parameters to the orchestration layer. Listing 5.1 illustrates the central part of this process.

```
1 config = load_config(args.project, args.config)
2
3 selected_smells = _resolve_smells(args, config)
4
5 repo_name = _pick(args.repo, config.get("repo"))
6 dirty_waters_backend = _pick(
7     args.dirty_waters_backend,
8     config.get("dirty_waters_backend"),
9     "disabled",
10 )
11 wsl_distro = _pick(
12     args.wsl_distro,
13     config.get("wsl_distro"),
14     "Ubuntu",
15 )
16 dirty_waters_root = _pick(
17     args.dirty_waters_root,
18     config.get("dirty_waters_root"),
19     "/home/parafita/dirty-waters",
20 )
21 unmaintained_threshold_months = _pick(
22     args.unmaintained_threshold_months,
23     config.get("unmaintained_threshold_months"),
24     24,
25 )
26
27 requires_dirty_waters = any(
28     smell in DIRTY_WATERS_SMELLS for smell in selected_smells
29 )
30
31 if requires_dirty_waters and not repo_name:
32     raise RuntimeError(
33         "Dirty-Waters-based smells require a GitHub repository
34         identifier. "
35         "Provide it with --repo or in the configuration file."
36     )
37
38 output_dir = run_analysis(
39     project_path=args.project,
40     selected_smells=selected_smells,
41     repo_name=repo_name,
42     dirty_waters_backend=dirty_waters_backend,
```

```

42     wsl_distro=wsl_distro,
43     dirty_waters_root=dirty_waters_root,
44     unmaintained_threshold_months=unmaintained_threshold_months,
45 )

```

Listing 5.1: Resolution of execution parameters in the CLI layer

This excerpt illustrates two relevant implementation concerns. First, it shows the precedence mechanism between command-line arguments and configuration values. Second, it shows that validation of execution constraints is performed before the analysis itself begins. This avoids unnecessary execution in cases where mandatory conditions, such as a repository identifier for Dirty-Waters-based checks, are not satisfied.

5.3.2 Registry-Based Smell Dispatch

A second important implementation detail is the use of a registry-based dispatch mechanism for smell execution. Rather than hardcoding the execution flow of each smell individually, the prototype keeps a central smell registry and resolves detectors dynamically from their names. This design keeps the orchestration layer generic and makes the addition of new smells considerably simpler.

Listing 5.2 shows the central execution loop of the orchestration layer.

```

1  context = {
2      "project_path": str(project_dir),
3      "package_json": package_json,
4      "package_lock": package_lock,
5      "repo_name": repo_name,
6      "dirty_waters_backend": dirty_waters_backend,
7      "wsl_distro": wsl_distro,
8      "dirty_waters_root": dirty_waters_root,
9      "unmaintained_threshold_months": unmaintained_threshold_months
10     ,
11 }
12 for smell in result.selected_smells:
13     detector = SMELL_REGISTRY.get(smell)
14
15     if detector is None:
16         result.errors.append(f"Unsupported smell requested: '{smell}'")
17         continue
18
19     try:
20         findings = detector.detect(**context)
21         result.findings.extend(findings)

```

```

22     except Exception as exc:
23         result.errors.append(f"{smell}: {exc}")

```

Listing 5.2: Registry-based execution of smell detectors

This implementation detail is particularly relevant because it decouples the orchestration layer from smell-specific logic. The orchestrator is only responsible for preparing the shared execution context and dispatching the selected detectors. As a consequence, the implementation remains open to extension while preserving a uniform execution model for all smell categories.

5.3.3 Shared Structure for Dirty-Waters-Based Smells

Although the native detectors are independent, the Dirty-Waters-based smells share a common execution pattern. To avoid repeating nearly identical integration logic across multiple smells, a shared helper was introduced. Each Dirty-Waters-based smell is described through a small configuration entry containing severity, external flags, summary patterns, and a message template.

Listing 5.3 presents a representative excerpt of this configuration-driven approach.

```

1  DIRTY_WATERS_SMELL_CONFIG = {
2      "deprecated-dependency": {
3          "severity": "medium",
4          "flags": ["--check-deprecated"],
5          "summary_patterns": [
6              r"Packages that are deprecated .*\s*(\d+)",
7          ],
8          "message": "Dirty-Waters reported deprecated dependencies.
9              ",
10     },
11     "no-source-code-link": {
12         "severity": "high",
13         "flags": ["--check-source-code"],
14         "summary_patterns": [
15             r"Packages .*source code repositior.*\s*(\d+)",
16         ],
17         "message": "Dirty-Waters reported dependencies with
18             missing or invalid source code repository links.",
19     },
20     "aliased-packages": {
21         "severity": "low",
22         "flags": ["--check-aliased-packages"],
23         "summary_patterns": [
24             r"Packages .*aliased.*\s*(\d+)",
25         ],

```

```

24         "message": "Dirty-Waters reported aliased packages.",
25     },
26 }

```

Listing 5.3: Configuration-driven definition of Dirty-Waters-based smells

This configuration-driven design reduced code duplication and made the implementation of new Dirty-Waters-based smells more systematic. Instead of creating a fully distinct execution path for each smell, the implementation only needs to specify the relevant external check flags and the report patterns used to extract the summary count from the generated report.

5.3.4 Knip Integration

The integration with Knip was implemented through a dedicated adapter whose responsibility is to locate an executable Knip command, invoke it with the appropriate arguments, and convert its JSON output into the internal finding model used by the prototype. This approach avoids coupling the smell detectors directly to subprocess execution and keeps the parsing logic isolated in a single place.

Listing 5.4 shows the core of this adapter.

```

1  cmd = _find_knip_command(project_path)
2  if cmd is None:
3      errors.append(
4          "Knip is not available for this project. "
5          "Install it locally in the target project, or make sure "
6          "'knip' or 'npx' is available on PATH."
7      )
8      return findings, errors
9
10 full_cmd = cmd + ["--dependencies", "--reporter", "json"]
11
12 result = subprocess.run(
13     full_cmd,
14     cwd=project_path,
15     capture_output=True,
16     text=True,
17     check=False,
18 )
19
20 if result.returncode not in (0, 1):
21     stderr = result.stderr.strip()
22     stdout = result.stdout.strip()
23     details = stderr if stderr else stdout if stdout else "no
        diagnostic output"

```

```
24     errors.append(  
25         f"Knip returned unexpected exit code {result.returncode}:  
           {details}"  
26     )  
27     return findings, errors  
28  
29 payload = json.loads(result.stdout.strip())
```

Listing 5.4: Core execution flow of the Knip adapter

This adapter illustrates a broader implementation pattern adopted throughout the prototype: external tools are not treated as opaque command invocations, but as structured integrations that must validate availability, normalize outputs, and handle failure conditions explicitly. This made the Knip-based smells easier to test through mocks and easier to integrate into the same execution flow as native smells.

5.3.5 Dirty-Waters Integration

The Dirty-Waters integration required a more elaborate implementation due to its stronger environmental dependencies. In the tested project context, this integration was especially sensitive on Windows, where execution had to be mediated through WSL/Ubuntu. For this reason, the adapter was designed to support multiple execution backends and to manage token propagation, debug artifact persistence, report path extraction, and result recovery.

Listing 5.5 shows a representative excerpt of the adapter logic.

```
1 def _resolve_backend(backend: str) -> str:  
2     if backend == "auto":  
3         return "wsl" if os.name == "nt" else "native"  
4     return backend  
5  
6 backend = _resolve_backend(backend)  
7  
8 if backend == "disabled":  
9     errors.append(  
10         "Dirty-Waters backend is disabled. "  
11         "Use 'wsl', 'native', or 'auto' to enable Dirty-Waters-  
           based analysis."  
12     )  
13     return None, errors  
14  
15 if backend not in {"wsl", "native"}:  
16     errors.append(  
17         f"Unsupported Dirty-Waters backend: '{backend}'. "  
18         "Supported values are: disabled, wsl, native, auto."  
19     )
```

```
20     return None, errors
21
22     github_token = os.getenv("GITHUB_API_TOKEN", "").strip()
23     if not github_token:
24         errors.append(
25             "GITHUB_API_TOKEN is not set. Dirty-Waters requires a
                GitHub token in the environment."
26         )
27     return None, errors
```

Listing 5.5: Backend resolution and validation in the Dirty-Waters adapter

A second relevant aspect of this integration is the recovery of the analysis report produced by Dirty-Waters. Since the report may be generated inside WSL and not always copied back to the local filesystem, the adapter was implemented to resolve the Linux-side report path, transform it into a user-accessible location, and return it together with a structured execution note whenever possible. This was an important practical improvement, because it allowed the prototype to treat the analysis as successful even when the report was not copied back automatically, provided that the generated report location could still be identified.

5.3.6 Configuration Profiles

Another practical implementation decision was to include reusable configuration profiles for common execution scenarios. In particular, separate example configurations were created for default local/Knip-based analysis and for Dirty-Waters-based analysis. This improved usability and reduced the burden of repeatedly passing long command-line invocations, especially during testing and manual validation.

Listing 5.6 shows a representative configuration profile for Dirty-Waters-based execution.

```
1 {
2     "smells": ["dirty-waters-all"],
3     "repo": "owner/repository",
4     "dirty_waters_backend": "wsl",
5     "wsl_distro": "Ubuntu",
6     "dirty_waters_root": "/home/your-user/dirty-waters"
7 }
```

Listing 5.6: Example configuration for Dirty-Waters-based smells

This configuration mechanism complements the command-line interface by making executions more reproducible and less error-prone. It was particularly useful in

the Dirty-Waters case, where multiple backend-related parameters had to be provided consistently across executions.

5.4 Requirements Coverage and Validation

The implementation and validation of the proposed tool were guided by the requirements defined in Section 3.8. In order to make this relationship explicit, this section presents a traceability view between the functional requirements, the implemented components, and the validation evidence available in the project repository.

The goal of this traceability analysis is not only to show that each requirement was addressed by the implementation, but also to clarify how each capability was validated. Since the tool combines native detectors, external-tool adapters, configuration mechanisms, and output generation, different requirements are validated through different forms of evidence, including unit tests, integration tests, mock-based tests, static inspection, and execution over prepared npm projects.

Table 5.1 summarizes the coverage of the functional requirements by the implemented components and by the corresponding validation evidence.

Table 5.1: Traceability between functional requirements, implementation evidence, and validation evidence.

Req.	Requirement summary	Implementation evidence	Validation evidence
FR1	Analyze npm projects.	Command-line workflow and <code>run_analysis</code> function.	Integration tests over prepared npm projects, including clean, mixed, duplicate-version, installation-script, and Knip-based samples.
FR2	Resolve analysis configuration.	Configuration loading through <code>config.py</code> , command-line parameters, and default configuration discovery.	Execution scenarios using default parameters and configuration-driven analysis.
FR3	Select smells for analysis.	Smell selection through supported smell identifiers and <code>SMELL_REGISTRY</code> .	Unit and integration tests executing selected detectors and selected-smell workflows.
FR4	Inspect dependency manifests.	<code>manifest_reader.py</code> and manifest-based detectors that inspect <code>package.json</code> .	Unit tests for pinned dependency, URL dependency, restrictive constraint, permissive constraint, and unmaintained package detection.
FR5	Inspect dependency lockfiles.	<code>lockfile_reader.py</code> and lockfile-based duplicate-version detector.	Unit and integration tests for duplicate resolved versions and lockfile-related behavior.
FR6	Detect local dependency smells.	Native detectors under <code>src/smells/</code> for local dependency-management indicators.	Unit tests for native detectors and integration tests over prepared npm projects.
FR7	Detect usage-consistency smells.	Knip-based detectors and <code>knip_adapter.py</code> .	Mock-based Knip adapter tests and integration test for Knip-based findings.
FR8	Detect ecosystem and provenance smells.	Dirty-Waters-based detectors and <code>dirty_waters_adapter.py</code> .	Mock-based Dirty-Waters adapter tests covering successful execution, disabled backend, missing token, subprocess failures, and report recovery scenarios.

Continued on next page

Req.	Requirement summary	Implementation evidence	Validation evidence
FR9	Normalize detected findings.	<code>Finding</code> dataclass and common finding fields: <code>smell</code> , <code>dependency</code> , <code>severity</code> , <code>evidence</code> , and <code>message</code> .	Unit, mock-based, and integration tests checking the produced finding structures.
FR10	Generate persistent output artifacts.	<code>report_writer.py</code> , generating <code>findings.json</code> , <code>errors.json</code> , and <code>report.txt</code> .	Integration tests validating output artifact generation after analysis execution.
FR11	Report execution errors.	Error collection through <code>AnalysisResult</code> and adapter-level error handling.	Mock-based tests for missing tools, invalid JSON output, unexpected return codes, missing GitHub token, disabled Dirty-Waters backend, and subprocess failures.
FR12	Explain detected findings.	Natural-language <code>message</code> field in each <code>Finding</code> and human-readable <code>report.txt</code> .	Unit and integration tests checking generated findings and manually inspected report outputs.

The traceability table shows that all functional requirements are addressed by at least one implemented component and by at least one form of validation evidence. Requirements related to local analysis are mainly covered by unit tests and prepared npm project integration tests. Requirements involving external tools are primarily validated through mock-based tests, since Knip and Dirty-Waters depend on external executables, environment configuration, and metadata availability.

In addition to the requirement-level mapping, the automated test suite provides evidence that the implemented workflow behaves consistently across the main analysis layers. The final validation run executed 39 automated tests, covering unit tests, integration tests, and mock-based adapter tests. These tests validate detector correctness, report generation, adapter error handling, and the ability to generate partial results when external analysis components fail.

Table 5.2 summarizes the main groups of tests included in the validation process.

Table 5.2: Summary of automated tests used to validate the implemented tool.

Test group	Number of tests	Main validation purpose
Unit tests	20	Validate the behavior of native detectors using controlled synthetic inputs and positive/negative cases.
Integration tests	5	Validate the complete analysis workflow over prepared npm projects, including detector execution and output artifact generation.
Mock-based tests	14	Validate Knip and Dirty-Waters adapter behavior under successful execution, unavailable tools, invalid outputs, missing configuration, and subprocess failure scenarios.
Total	39	Provide automated validation evidence for detector correctness, pipeline execution, external-tool integration, and failure handling.

Overall, the validation results indicate that the prototype satisfies the functional requirements defined for this stage of the project. The tests confirm that the tool can analyze npm projects, execute different detection mechanisms, normalize the resulting findings, generate persistent reports, and handle external-tool failures without preventing report generation. Nevertheless, some quality attributes, such as performance efficiency over larger real-world projects and broader maintainability assessment, remain dependent on the evaluation conditions and should be interpreted within the scope of the implemented prototype.

5.5 Testing and Validation

The implemented prototype was validated through automated tests covering three complementary perspectives: unit testing, integration testing, and mock-based testing. This combination was adopted to provide confidence both in the internal behavior of the smell detectors and in the correctness of the end-to-end execution flow.

Unit tests were used to validate the local detection logic in isolation, especially for detectors whose behavior can be expressed as pure or near-pure functions over parsed project artifacts. Integration tests were then used to validate the complete pipeline over prepared sample projects, covering the interaction between input loading, orchestration, smell dispatch, and report generation. Finally, mock-based tests were introduced for external tool integrations, making it possible to validate the parsing and error-handling logic of Knip and Dirty-Waters without depending on their actual installation in every test scenario.

This testing strategy was particularly important given the hybrid nature of the prototype. Since the implementation combines local logic with subprocess-based integrations, no single test style would be sufficient on its own. The adopted approach therefore aimed to balance isolation, realism, and reproducibility.

5.5.1 Unit Tests

Unit tests were created for the native smell detectors, focusing on their expected behavior for both positive and negative cases. Since these detectors operate directly over local manifest or lockfile data, they are especially suitable for isolated automated testing. This made it possible to validate the implemented rules without requiring a complete project execution or the presence of external tools.

The tests were implemented with `pytest` and organized under the `tests/unit` directory. Each test file targets a specific smell category or detector, making the suite easier to maintain and understand. Examples include tests for pinned dependencies, URL-based dependencies, restrictive and permissive version constraints, duplicate resolved versions, installation scripts, missing package locks, and the unmaintained-package heuristic.

Listing 5.7 shows a representative example of a unit test for the pinned-dependency smell.

```
1 def test_pinned_dependency_detects_exact_version():
2     detector = PinnedDependencyDetector()
3
4     findings = detector.detect(
5         package_json={
6             "dependencies": {
```

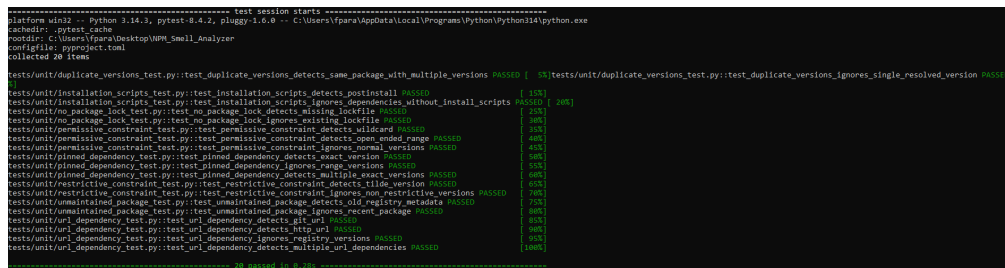
```

7         "react": "18.2.0",
8         "lodash": "^4.17.21",
9     }
10 },
11     package_lock=None,
12     project_path="dummy_project",
13 )
14
15 assert len(findings) == 1
16 assert findings[0].smell == "pinned-dependency"
17 assert findings[0].dependency == "react"

```

Listing 5.7: Representative unit test for a native smell detector

This type of test is useful because it directly validates the intended rule encoded in the detector and documents the expected behavior in an executable form. In practice, the unit test suite provided fast feedback during implementation and refactoring of the native smells.



```

===== test session starts =====
platform win32 -- Python 3.14.1, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\jparal\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\jparal\Desktop\NPM_Smell_Analyzer
configfile: pyproject.toml
collected 20 items

tests/unit/duplicate_versions_test.py::test_duplicate_versions_detects_same_package_with_multiple_versions PASSED [ 50%]
tests/unit/duplicate_versions_test.py::test_duplicate_versions_ignores_single_resolved_version PASSED [ 60%]
tests/unit/installation_scripts_test.py::test_installation_scripts_detects_postinstall PASSED [ 70%]
tests/unit/installation_scripts_test.py::test_installation_scripts_ignores_dependencies_without_install_scripts PASSED [ 80%]
tests/unit/no_package_lock_test.py::test_no_package_lock_detects_missing_lockfile PASSED [ 90%]
tests/unit/no_package_lock_test.py::test_no_package_lock_ignores_existing_lockfile PASSED [100%]
tests/unit/permissive_constraint_test.py::test_permissive_constraint_detects_wildcard PASSED [ 10%]
tests/unit/permissive_constraint_test.py::test_permissive_constraint_detects_open_version_range PASSED [ 20%]
tests/unit/permissive_constraint_test.py::test_permissive_constraint_ignores_normal_versions PASSED [ 30%]
tests/unit/pinned_dependency_test.py::test_pinned_dependency_detects_exact_version PASSED [ 40%]
tests/unit/pinned_dependency_test.py::test_pinned_dependency_ignores_range_versions PASSED [ 50%]
tests/unit/pinned_dependency_test.py::test_pinned_dependency_detects_multiple_exact_versions PASSED [ 60%]
tests/unit/restrictive_constraint_test.py::test_restrictive_constraint_ignores_non_restrictive_versions PASSED [ 70%]
tests/unit/restrictive_constraint_test.py::test_restrictive_constraint_detects_exact_version PASSED [ 80%]
tests/unit/unmaintained_package_test.py::test_unmaintained_package_ignores_recent_packages PASSED [ 90%]
tests/unit/url_dependency_test.py::test_url_dependency_detects_git_url PASSED [100%]
tests/unit/url_dependency_test.py::test_url_dependency_detects_https_url PASSED [ 10%]
tests/unit/url_dependency_test.py::test_url_dependency_ignores_registry_versions PASSED [ 20%]
tests/unit/url_dependency_test.py::test_url_dependency_detects_multiple_url_dependencies PASSED [ 30%]

```

Figure 5.1: Successful execution of the unit test suite.

5.5.2 Integration Tests

While unit tests validate detectors in isolation, integration tests were used to validate the full analysis flow over prepared sample projects. For this purpose, a set of small npm projects was created under the `test_projects` directory, each one designed to expose specific smell scenarios or combinations of smells.

These projects made it possible to test the interaction between project file loading, orchestration, smell resolution, detection, and output generation. This approach provided stronger confidence that the prototype behaves correctly under realistic execution conditions, rather than only under isolated detector-level assumptions.

The integration tests were implemented under `tests/integration` and invoked the analysis pipeline programmatically. Listing 5.8 shows a representative example.

```

1 def test_sample_mixed_detects_expected_smells():
2     output_dir = run_analysis(
3         project_path="test_projects/sample_mixed",

```

```

4         selected_smells=[
5             "url-dependency",
6             "pinned-dependency",
7             "restrictive-constraint",
8         ],
9     )
10
11     with open(Path(output_dir) / "findings.json", "r", encoding="
        utf-8") as f:
12         findings = json.load(f)
13
14     smell_names = [finding["smell"] for finding in findings]
15
16     assert "url-dependency" in smell_names
17     assert "pinned-dependency" in smell_names
18     assert "restrictive-constraint" in smell_names

```

Listing 5.8: Representative integration test over a prepared sample project

This category of tests was especially relevant because it exercises the prototype in a way that is close to actual use: a project is analyzed, the selected smells are executed, and the produced artifacts are inspected to confirm the expected behavior.

```

Platform: win32 -- Python 3.10.3, pytest-6.4.2, pluggy-1.0.0 -- C:\Users\para\AppData\Local\Programs\Python\Python310\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\para\AppData\Local\Temp\pip-installation\python.exe
configfile: pyproject.toml
collected 5 items

tests/integration/sample_clean_test.py::test_sample_clean_has_no_local_findings PASSED
tests/integration/sample_duplicate_versions_test.py::test_sample_duplicate_versions_detects_duplicate_versions PASSED
tests/integration/sample_installation_scripts_test.py::test_sample_installation_scripts_detects_installation_scripts PASSED
tests/integration/sample_knip_test.py::test_sample_knip_detects_knip_smells PASSED
tests/integration/sample_mixed_test.py::test_sample_mixed_detects_expected_smells PASSED

===== 5 passed in 10.00s =====

```

Figure 5.2: Successful execution of the integration test suite over prepared sample projects.

5.5.3 Mock-Based Tests

A third category of automated tests was introduced to validate the integrations with external tools. Since Knip and Dirty-Waters are executed through subprocesses and may depend on environment-specific conditions, tests based exclusively on real executions would be more fragile and harder to reproduce. To address this, mock-based tests were implemented using `unittest.mock.patch`.

These tests simulate the behavior of subprocess execution and command discovery, allowing the prototype to validate how adapter outputs are interpreted and how execution failures are reported. This was particularly useful for checking scenarios such as invalid JSON returned by Knip, unexpected subprocess return codes, unavailable commands, missing tokens, and Dirty-Waters execution failures.

Listing 5.9 presents a representative mock-based test for the Knip adapter.

```

1 @patch("src.external_tools.knip_adapter._find_knip_command")
2 @patch("src.external_tools.knip_adapter.subprocess.run")
3 def test_knip_adapter_parses_unused_dependency(mock_run,
4         mock_find_command, tmp_path):
5     mock_find_command.return_value = ["npx", "knip"]
6     mock_run.return_value = SimpleNamespace(
7         returncode=1,
8         stdout="",
9         {
10             "issues": [
11                 {
12                     "file": "package.json",
13                     "dependencies": [
14                         {
15                             "name": "lodash",
16                             "line": 5,
17                             "col": 6
18                         }
19                     ]
20                 }
21             ]
22         },
23         stderr="",
24     )
25
26     findings, errors = run_knip_dependency_analysis(str(tmp_path))
27
28     assert errors == []
29     assert len(findings) == 1
30     assert findings[0].smell == "unused-dependency"
31     assert findings[0].dependency == "lodash"

```

Listing 5.9: Representative mock-based test for an external tool
adapter

Mock-based tests strengthened the robustness of the implementation by validating external integration logic independently of the actual presence of the integrated tools. This was especially important in the case of Dirty-Waters, whose execution depends on more specific runtime conditions and operating-system support.

```

platform win32 -- Python 3.14.3, pytest-8.4.2, pluggy-1.6.0 -- C:\Users\fpapa\AppData\Local\Programs\Python\Python314\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\fpapa\Desktop\UPM_Smell_Analyzer
configfile: pyproject.toml
collected 14 items

tests/mocks/dirty_waters_adapter_mock_test.py::test_dirty_waters_adapter_returns_error_when_backend_is_disabled PASSED [ 71%]
tests/mocks/dirty_waters_adapter_mock_test.py::test_dirty_waters_adapter_returns_error_when_token_is_missing PASSED [ 86%]
tests/mocks/dirty_waters_adapter_mock_test.py::test_dirty_waters_adapter_success_with_report_copied_back PASSED [ 100%]
tests/mocks/dirty_waters_adapter_mock_test.py::test_dirty_waters_adapter_success_with_report_copied_back_with_report_path PASSED [ 100%]
tests/mocks/dirty_waters_adapter_mock_test.py::test_dirty_waters_adapter_returns_error_when_subprocess_fails PASSED [ 100%]
tests/mocks/dirty_waters_adapter_mock_test.py::test_dirty_waters_adapter_returns_error_when_report_path_cannot_be_determined PASSED [ 100%]
tests/mocks/knip_adapter_mock_test.py::test_knip_adapter_parses_unused_dev_dependency PASSED [ 100%]
tests/mocks/knip_adapter_mock_test.py::test_knip_adapter_parses_missing_dependency PASSED [ 100%]
tests/mocks/knip_adapter_mock_test.py::test_knip_adapter_returns_no_findings_when_issues_are_empty PASSED [ 100%]
tests/mocks/knip_adapter_mock_test.py::test_knip_adapter_returns_error_when_knip_command_not_found PASSED [ 100%]
tests/mocks/knip_adapter_mock_test.py::test_knip_adapter_returns_error_on_invalid_json PASSED [ 100%]
tests/mocks/knip_adapter_mock_test.py::test_knip_adapter_returns_error_when_subprocess_fails PASSED [ 100%]
tests/mocks/knip_adapter_mock_test.py::test_knip_adapter_returns_error_for_unexpected_return_code PASSED [ 100%]

===== 14 passed in 0.72s =====

```

Figure 5.3: Successful execution of the mock-based test suite for external tool adapters.

5.5.4 Requirements Traceability

While Table 5.1 maps the functional requirements to the corresponding implementation and validation evidence, Table 5.3 focuses specifically on the validation of the non-functional requirements. This separation avoids duplicating the functional requirement mapping and makes the quality-related evidence explicit, including performance measurements, reproducibility checks, static security inspection, maintainability metrics, and clean installation validation.

Table 5.3: Traceability between non-functional requirements and validation evidence.

Req.	Quality Attribute	Validation Method	Measured / Observed Result	Status
NFR1	Usability and structured reporting.	Generated <code>findings.json</code> files were inspected to verify that each finding contains the mandatory fields <code>smell</code> , <code>dependency</code> , <code>severity</code> , <code>evidence</code> , and <code>message</code> . The generated <code>report.txt</code> files were also inspected.	All inspected findings contained the required fields and developer-oriented messages. The generated <code>report.txt</code> files summarized the selected smells, total findings, total errors, and detected findings in a readable form.	Satisfied
NFR2	Performance efficiency.	Execution time was measured over a prepared npm test project. Local static analysis was measured separately from the default workflow, since external integrations and metadata-dependent checks may introduce environment-dependent execution time.	Local static analysis completed in approximately 0.397 seconds . The default local/Knip workflow was executed three times, with execution times of approximately 14.005 , 10.438 , and 11.079 seconds , resulting in an average of approximately 11.84 seconds .	Satisfied
NFR3	Fault tolerance and partial-result reporting.	Mock-based tests were executed for the Knip and Dirty-Waters adapters, covering unavailable tools, invalid outputs, missing credentials, disabled backend, unexpected return codes, subprocess failures, and report recovery scenarios.	The mock-based test suite validated that external-tool failures are recorded as errors without preventing the remaining analysis workflow from producing output artifacts whenever partial reporting is possible.	Satisfied
NFR4	Reproducibility.	The same controlled analysis was executed three consecutive times over the same prepared project and selected smell set. The resulting <code>findings.json</code> files were compared using SHA-256 hashes.	The three executions produced identical <code>findings.json</code> hashes. The number of unique hashes was 1 , indicating bit-by-bit identical results for the tested scenario.	Satisfied

NFR5	Security of native analysis.	Static inspection was performed over <code>src/smells/</code> to verify that native detectors do not execute dependency code and do not contain subprocess execution, <code>exec()</code> , <code>eval()</code> , dynamic imports, or similar execution mechanisms. A second inspection verified where subprocess execution is used in the codebase.	No dependency-code execution mechanisms were found inside native detector modules. Subprocess execution was found only in external-tool adapter modules, namely the Knip and Dirty-Waters adapters.	Satisfied
NFR6	Maintainability and extensibility.	Radon was executed over the <code>src/</code> directory to measure cyclomatic complexity and Maintainability Index. The detector architecture was also inspected to verify that native smells are modular and registered through the smell registry.	Radon analyzed 73 classes, functions, and methods. The average cyclomatic complexity was A (4.85) , which is below the defined threshold of 10. A total of 7 out of 73 analyzed blocks were graded C or worse, corresponding to approximately 9.59% , below the 15% threshold. The Maintainability Index result was grade A for all analyzed modules . The architecture also supports adding new native detectors through a dedicated module and registry entry.	Satisfied
NFR7	Low setup complexity.	The package was built and installed from a generated wheel in a clean virtual environment. Imports were tested outside the repository directory, and the CLI entry point was invoked using <code>npm-smell-analyzer -help</code> .	The clean wheel installation succeeded, core modules were importable outside the repository, and the CLI entry point was available. The command-line help was displayed successfully.	Satisfied

Overall, the traceability table shows that the validation strategy covers the three main levels of the prototype: native detector logic, complete analysis execution over prepared npm projects, and controlled simulation of external-tool behavior. This is especially relevant because the tool combines local static analysis with external integrations, meaning that no single test category would be sufficient to validate the complete solution.

5.6 Repository Artifacts and Reproducibility

An important outcome of the implemented prototype is not only the tool itself, but also the set of repository artifacts [31] that support its understanding, execution, and validation. In this project, the repository was treated as part of the final contribution, rather than as a mere storage location for source code. This decision is consistent with the practical nature of the prototype and with the objective of making the implemented solution inspectable and reusable by other developers or evaluators.

The repository includes the main source code of the prototype under the `src` directory, together with several complementary artifacts that support reproducibility. These include automated tests organized by category, prepared sample projects used in integration testing, example configuration files for typical execution modes, and a `README.md` file documenting installation, execution, and current limitations. As a result, the repository provides not only the implementation itself, but also the materials required to understand how the prototype is expected to be used and validated.

From the perspective of execution reproducibility, the prototype was designed to be installable and executable through a standard Python workflow. The command-line entry point, the configuration mechanism, and the reporting artifacts were all implemented in a way that makes repeated execution more systematic. In particular, the use of configuration files reduces ambiguity in repeated runs and makes it easier to preserve execution conditions across different test scenarios.

Reproducibility was also considered in the validation process. The repository contains unit tests for native smells, integration tests based on prepared npm sample projects, and mock-based tests for the Knip and Dirty-Waters adapters. This means that a significant portion of the prototype behavior can be re-evaluated without reconstructing the testing setup from scratch. In this sense, the repository captures not only the implementation logic, but also the validation strategy associated with it.

Nevertheless, reproducibility is not absolute in every execution mode. Some parts of the prototype depend on external tools and environment-specific conditions.

In particular, Dirty-Waters-based analyses remain more dependent on operating-system support, backend configuration, and the availability of external credentials such as a GitHub token. Similarly, certain local smells require the analyzed project to contain installed dependencies in `node_modules`, and metadata-based checks may depend on external package registry information. These aspects do not invalidate the reproducibility of the prototype, but they do define practical limits that must be acknowledged.

Overall, the repository acts as an extension of the implemented solution itself. It documents the structure of the prototype, preserves the supporting artifacts needed for execution and testing, and strengthens the value of the work as a reusable academic and technical contribution.

5.7 Main Development Decisions and Trade-Offs

The implementation of *NPM Smell Analyzer* involved a number of practical design decisions shaped by the intended scope of the prototype. Since the objective was not to build a full industrial platform, but rather a lightweight and academically grounded analysis tool, several trade-offs had to be made between completeness, robustness, portability, and implementation effort.

A first important decision was to separate the default local execution flow from the Dirty-Waters-based execution flow. Native smells and Knip-based smells can be executed quickly and depend mostly on local project artifacts, making them suitable for the default `-all` mode. Dirty-Waters-based smells, in contrast, depend on repository-level metadata, external credentials, and more restrictive execution conditions. For this reason, they were intentionally removed from the default fast path and exposed through dedicated execution modes. This decision improved responsiveness and usability for common analyses, at the cost of requiring a separate execution path for richer repository-level checks.

A second decision concerned the introduction of configuration files. Early versions of the prototype relied primarily on command-line arguments, which quickly became cumbersome, especially for Dirty-Waters-based analyses that require multiple backend-related parameters. The adoption of configuration files improved repeatability and reduced invocation complexity, while preserving flexibility through command-line overrides. The main trade-off was the need to maintain an additional configuration layer, but this cost was considered acceptable given the usability gains obtained.

Another relevant decision was the encapsulation of external tools behind dedicated adapters. Rather than allowing Knip and Dirty-Waters logic to spread across the orchestration and smell-detection layers, both integrations were isolated behind adapter modules. This made the codebase cleaner and easier to test, especially

through mock-based tests. The corresponding trade-off is that the architecture introduces an additional abstraction layer, which slightly increases structural complexity, but greatly improves maintainability and separation of concerns.

The implementation of the unmaintained-package smell also reflects a deliberate trade-off. Instead of attempting to infer package maintenance status through a more complex and potentially ambiguous ecosystem-wide analysis, the prototype adopts a lightweight heuristic based on the elapsed time since the last npm registry modification. This choice made the smell feasible within the scope of the prototype, but it also means that the result should be interpreted as an indicator rather than as definitive evidence of abandonment.

A similar trade-off applies to the installation-scripts smell. The implemented detector inspects installed dependency manifests in `node_modules` in order to identify lifecycle scripts such as `preinstall`, `install`, and `postinstall`. This provides a practical local mechanism for detecting potentially relevant signals, but it also makes the smell dependent on the presence of installed dependencies in the analyzed project. In other words, the prototype favors a direct and lightweight implementation over an approach that would try to reconstruct such information from remote metadata alone.

The implementation scope also required explicit exclusion decisions. Some weak-link indicators identified during the design phase, such as overloaded maintainers, too many contributors, too many maintainers, and expired maintainer domains, were not implemented in the prototype. Although these indicators are relevant from a supply-chain perspective, their detection would require broader ecosystem metadata, more complex threshold calibration, or more fragile assumptions than those considered appropriate for the current prototype. Excluding them was therefore a conscious scope-management decision rather than an omission caused by oversight.

Finally, the prototype reflects a general trade-off between portability and practical tool reuse. Reusing Knip and Dirty-Waters allowed the solution to benefit from specialized external analyses instead of reimplementing them from scratch. However, this also introduced dependence on external runtime conditions and tool availability. In particular, Dirty-Waters remained more sensitive to operating-system differences and environment setup than the rest of the prototype. The adopted design therefore privileges practical integration and academic demonstrability over full environment independence.

Overall, these decisions reflect the intended positioning of the prototype: a lightweight, modular, and reproducible analysis tool that supports a meaningful subset of software supply chain smells for npm-based projects, while explicitly acknowledging the technical and scope-related limits that accompany such a design.

Chapter 6

Conclusions

This chapter aims to expose the achieved goals, some of the difficulties faced during the development of this project and some suggestions for possible future development. In the end of this chapter is given a final appreciation.

6.1 Achieved Objectives

The main objective of this project was to study how selected software supply chain smells in npm-based projects could be detected through a practical and reproducible prototype. This objective was achieved through the design and implementation of *NPM Smell Analyzer*, a command-line tool that combines native local analysis, Knip-based dependency-usage checks, and Dirty-Waters-based repository-level analysis.

From a technical perspective, the project resulted in a working prototype capable of detecting a meaningful subset of smells derived from the literature studied in previous chapters. The implementation supports local smells such as pinned dependencies, URL dependencies, restrictive and permissive constraints, duplicate resolved versions, installation scripts, and a heuristic for potentially unmaintained packages. In addition, the prototype integrates Knip for detecting unused and missing dependencies, and Dirty-Waters for detecting repository-level supply chain smells such as deprecated dependencies, missing source-code references, missing source-code tags or SHAs, aliased packages, and other related indicators.

Another important objective was to ensure that the solution was not limited to a conceptual proposal, but instead materialized as an executable and testable artifact. This objective was also achieved. The final repository includes the full implementation of the prototype, example configuration files, prepared sample projects, and an automated testing suite covering native smell detectors, integration scenarios, and mocked external-tool adapters. As a result, the project delivers not only an implemented tool, but also a reproducible academic artifact.

A further objective of the work was to explore the practical feasibility of combining lightweight local analysis with external specialized tools. In this regard, the project confirmed that such a hybrid approach is both viable and useful. Local rules allow the prototype to provide fast feedback over npm project artifacts, while integrations with Knip and Dirty-Waters extend the analysis beyond what could be reasonably implemented through local parsing alone.

Overall, the project fulfilled its main purpose: to transform the studied supply chain smell concepts into a concrete prototype that can analyze npm-based projects in a structured and extensible way.

6.2 Difficulties

The main difficulties encountered during the project were related not to the basic native smell detectors, but to the integration of external tools and to the practical constraints of the execution environment.

The most significant challenge concerned Dirty-Waters integration. While the tool is valuable for detecting repository-level supply chain smells, its execution proved more fragile and environment-dependent than the rest of the prototype. In the tested context, Dirty-Waters did not behave reliably as a native Windows integration, which required the adoption of WSL/Ubuntu as the main practical execution path. This introduced additional concerns related to backend configuration, token propagation, filesystem paths, and report retrieval. Although these difficulties were mitigated through adapter logic and configuration files, they remained one of the most demanding aspects of the implementation.

Another challenge was balancing usability with flexibility in the command-line interface. As the project evolved, it became clear that relying exclusively on command-line arguments made some execution modes, especially Dirty-Waters-based ones, too cumbersome and error-prone. This led to the introduction of configuration files, which improved repeatability and reduced invocation complexity, but also added another layer of execution logic that had to be validated and documented carefully.

A further difficulty concerned scope management. The literature review identified more candidate smells than could be realistically implemented within the

constraints of the project. Some weak-link indicators, particularly those requiring broader ecosystem-wide contributor or maintainer metadata, were found to be too dependent on assumptions, thresholds, or external information sources for the intended prototype scope. As a result, it was necessary to define and preserve a realistic implementation boundary.

Finally, the project also required a careful testing strategy. Since the prototype combines local logic with subprocess-based integrations, validating the implementation required more than one testing style. This led to the development of complementary unit, integration, and mock-based tests, which improved confidence in the tool but also increased the implementation effort associated with validation.

6.3 Future Work

Although the implemented prototype achieved its main goals, several directions for future work can be identified.

A first line of future work would be to improve the portability and robustness of external-tool integration, especially in the case of Dirty-Waters. A more stable cross-platform execution model, or a better abstraction over environment-specific execution backends, would reduce the dependence on WSL-specific workflows and make the prototype easier to adopt in heterogeneous development environments.

A second line of future work concerns the expansion of smell coverage. Some weak-link indicators identified during the design stage were intentionally excluded from the prototype due to scope and feasibility constraints. Future work could revisit these smells and investigate whether more reliable detection mechanisms can be developed for indicators such as overloaded maintainers, too many contributors, too many maintainers, or expired maintainer domains.

Another relevant direction would be to enrich the reporting capabilities of the prototype. At present, the tool generates structured JSON outputs and a textual summary, which is sufficient for the current scope. However, future work could explore richer reporting formats, severity aggregation, ranking of findings, or export formats more directly aligned with other security and supply chain analysis workflows.

It would also be valuable to strengthen the integration with additional ecosystem tools. The current hybrid approach already demonstrates the usefulness of combining native rules with external analysis. Future versions of the prototype could incorporate further tools for package risk analysis, provenance checks, or SBOM-related inspection, provided that such integrations remain compatible with the lightweight scope of the solution.

Finally, future work may also consider broadening the scope beyond npm-based projects. Although the final prototype was intentionally focused on npm projects,

the architectural structure adopted in the implementation suggests that the same overall approach could be extended to other package ecosystems, provided that suitable artifact readers, smell mappings, and external integrations are available.

6.4 Final Remarks

Overall, the project resulted in a useful and coherent prototype that bridges the gap between the conceptual discussion of software supply chain smells and their practical detection in npm-based projects. The developed solution does not aim to be a complete industrial-grade analysis platform, but rather a lightweight, modular, and academically grounded prototype capable of demonstrating the feasibility of the proposed approach.

One of the most relevant outcomes of the work is the confirmation that software supply chain smell detection benefits from a hybrid strategy. Some indicators can be handled effectively through local static analysis, while others are better addressed through dedicated external tools and repository-level metadata. The implementation of *NPM Smell Analyzer* showed that these heterogeneous detection mechanisms can be combined within a single command-line workflow while preserving a relatively clean architecture and a consistent internal data model.

The project also reinforced the importance of making academic prototypes reproducible. By organizing the implementation together with configuration examples, sample projects, automated tests, and documentation, the final repository becomes more than a code submission: it becomes a reusable artifact that can be inspected, executed, and extended.

In conclusion, the work delivered a practical contribution aligned with the original project objectives. It produced a working prototype, clarified the feasibility and limitations of the adopted detection strategies, and established a solid basis for future refinement and extension.

References

- [1] R. Liu, S. Bobadilla, B. Baudry, and M. Monperrus, “Dirty-Waters: Detecting software supply chain smells,” in *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, pp. 1045–1049, June 2025. [Cited on pages xi, 1, 13, 14, 15, 17, 20, 21, 22, 23, 24, 27, 29, 30, 31, 33, 51, 62, and 63]
- [2] “Advanced Persistent Threat compromise of Government Agencies, Critical Infrastructure, and Private Sector Organizations | CISA.” <https://www.cisa.gov/news-events/cybersecurity-advisories/aa20-352a>, Apr. 2021. [Cited on page 2]
- [3] “Cyber Resilience Act | Shaping Europe’s digital future.” <https://digital-strategy.ec.europa.eu/en/policies/cyber-resilience-act>. [Cited on page 2]
- [4] “A03 Software Supply Chain Failures - OWASP Top 10:2025.” https://owasp.org/Top10/2025/A03_2025-Software_Supply_Chain_Failures/. [Cited on page 2]
- [5] “The minimum elements for a software bill of materials (sbom) | national telecommunications and information administration.” <https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>. [Cited on pages 5 and 6]
- [6] “Dependency graph sbom - OWASP Cheat Sheet Series.” https://cheatsheetseries.owasp.org/cheatsheets/Dependency_Graph_SBOM_Cheat_Sheet.html. [Cited on pages 6 and 16]
- [7] “@cyclonedx/cyclonedx-npm.” <https://www.npmjs.com/package/@cyclonedx/cyclonedx-npm>, Mar. 2026. [Cited on pages 7 and 11]
- [8] “OWASP Dependency-Track | OWASP Foundation.” <https://owasp.org/www-project-dependency-track/>. [Cited on pages 7, 12, and 16]
- [9] “OWASP CycloneDX (ecma-424) | OWASP Foundation.” <https://owasp.org/www-project-cyclonedx/>. [Cited on pages 7, 8, 9, 11, and 15]
- [10] “CycloneDX specification.” <https://github.com/CycloneDX/specification>, Mar. 2026. [Cited on pages 7, 8, 9, 11, and 15]

-
- [11] “About semantic versioning - npm docs.” <https://docs.npmjs.com/about-semantic-versioning>. [Cited on page 9]
- [12] “package-lock.json - npm docs.” <https://docs.npmjs.com/cli/v8/configuring-npm/package-lock-json/>. [Cited on pages 9, 10, 20, 21, 22, 27, 28, 31, 52, and 62]
- [13] “Auditing package dependencies for security vulnerabilities - npm docs.” <https://docs.npmjs.com/auditing-package-dependencies-for-security-vulnerabilities/>. [Cited on pages 10, 20, 22, and 33]
- [14] “npm-audit - npm docs.” <https://docs.npmjs.com/cli/v9/commands/npm-audit>. [Cited on pages 10, 20, 22, and 33]
- [15] “Dependency-track | continuous sbom analysis platform.” <https://dependencytrack.org/>. [Cited on page 12]
- [16] “Dependency-track documentation.” <https://docs.dependencytrack.org/>. [Cited on pages 12 and 16]
- [17] “Cyclonedx tool center.” <https://cyclonedx.org/tool-center/>. [Cited on page 12]
- [18] C. Liu, S. Chen, L. Fan, B. Chen, Y. Liu, and X. Peng, “Demystifying the Vulnerability Propagation and its evolution via Dependency Trees in the NPM Ecosystem,” in *Proceedings of the 44th International Conference on Software Engineering*, pp. 672–684, May 2022. [Cited on pages 12, 13, 17, 20, 21, and 62]
- [19] A. J. Jafari, D. E. Costa, R. Abdalkareem, E. Shihab, and N. Tsantalis, “Dependency smells in javascript projects,” *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3790–3807, 2022. [Cited on pages 13, 14, 20, 21, 22, 23, 24, 27, 28, 31, 51, 53, and 63]
- [20] N. Zahan, T. Zimmermann, P. Godefroid, B. Murphy, C. Maddila, and L. Williams, “What are weak links in the npm supply chain?,” in *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*, pp. 331–340, May 2022. [Cited on pages 13, 14, 20, 21, 22, 23, 24, 27, 29, 31, 51, 62, and 63]
- [21] “chains-project/dirty-waters.” <https://github.com/chains-project/dirty-waters>, Feb. 2026. [Cited on pages 21 and 51]
- [22] “Specifying dependencies and devdependencies in a package.json file | npm docs.” <https://docs.npmjs.com/>

- `specifying-dependencies-and-devdependencies-in-a-package-json-file/`.
[Cited on pages 27, 28, and 31]
- [23] “npm view | npm docs.” <https://docs.npmjs.com/cli/v11/commands/npm-view>. [Cited on pages 29, 30, and 31]
- [24] “npm ci | npm docs.” <https://docs.npmjs.com/cli/v9/commands/npm-ci>.
[Cited on page 33]
- [25] “Iso/iec 25010 software quality model.” <https://iso25000.com/index.php/en/iso-25000-standards/iso-25010>. [Cited on page 36]
- [26] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1994.
[Cited on pages 47 and 48]
- [27] M. Fowler, “Registry.” <https://martinfowler.com/eaCatalog/registry.html>, 2003. [Cited on page 48]
- [28] “Unused dependencies | knip.” <https://knip.dev/typescript/unused-dependencies>. [Cited on pages 52 and 53]
- [29] “scripts | npm docs.” <https://docs.npmjs.com/cli/v8/using-npm/scripts/>. [Cited on page 52]
- [30] “depcheck/depcheck.” <https://github.com/depcheck/depcheck>. [Cited on page 53]
- [31] F. M. D. Parafita, “NPM Smell Analyzer: A tool for detecting software supply chain smells in npm projects.” https://github.com/Parafita123/NPM_Smell_Analyzer, 2026. GitHub Repository. [Cited on pages 65 and 90]